UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

PRIMER PARCIAL: SISTEMA DE CONTROL DE ASISTENCIA MEDIANTE QR PARA ESTUDIANTES

ESTUDIANTE: Loaiza Labarden Reymar

REGISTRO: 216027438

GRUPO: SA

MATERIA: Arquitectura del Software INF552

DOCENTE: Ing. Veizaga Flores Josué Obed

Santa Cruz de la Sierra - Bolivia

Semestre I/2026

Contenido

1. CAPTURA DE REQUISITOS 3
1.1. REQUISITOS FUNCIONALES 3
1.1.1. IDENTIFICAR ACTORES Y CASOS DE USO 3
1.1.2. DETALLE DE CASOS DE USO 3
1.1.3. DIAGRAMA GENERAL DE CASOS DE USO 9
1.2. REQUISITOS NO FUNCIONALES 9
2. ANALISIS 10
2.1. IDENTIFICACION DE MODULO 10
2.2. VISTA DE MODULOS 11
2.3. ENCAPSULAMIENTO DE CASOS DE USO 12
3. DISEÑO 13
3.1. DISEÑO DE LA ARQUITECTURA LOGICA 13
3.1.1. Módulo de Gestión 13
3.1.2. Módulo de Asistencia 13
3.2. DISEÑO DE LA BASE DE DATOS 14
3.2.1. DISEÑO CONCEPTUAL 14
3.2.2. DISEÑO LOGICO 15
3.2.3. DISEÑO FISICO 15
3.3. DISEÑO DE LA INTERFAZ 18
3.3.1. CU1 Gestión de Materias 18
3.3.2. CU2 Gestión de Grupos 19
3.3.3. CU3 Registrar estudiantes en un Grupo. 20
3.3.4. CU4 Gestionar Clases. 21
3.3.5. CU5 Generar código QR para la clase. 22
3.3.6. CU6 Registrar asistencia mediante QR. 22
3.4. DISEÑO DE DETALLE PROCEDIMENTAL 23
3.4.1. CU1 Gestión de Materias 23
3.4.2. CU2 Gestión de Grupos 25
3.4.3. CU3 Registrar estudiantes en un Grupo 27
3.4.4. CU4 Gestión de Clases 29
3.4.5. CU5 Generar código QR para la clase 31
3.4.6. CU6 Registrar asistencia mediante QR 33

1. CAPTURA DE REQUISITOS

1.1. REQUISITOS FUNCIONALES

1.1.1. IDENTIFICAR ACTORES Y CASOS DE USO

- Actores:

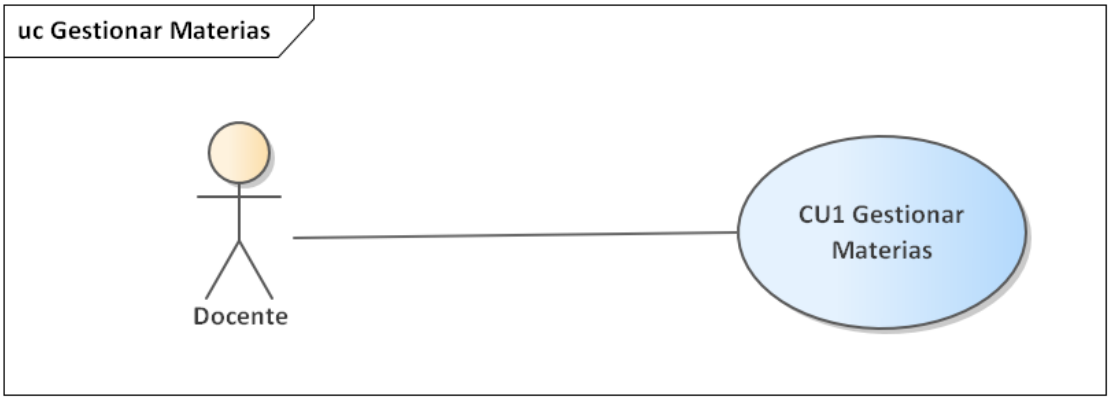
- Docente
- Estudiante

- Casos de uso:

- CU1: Gestionar Materias.
- CU2: Gestionar Grupos.
- CU3: Registrar estudiantes en un Grupo.
- CU4: Gestionar Clases.
- CU5: Generar código QR para la clase.
- CU6: Registrar asistencia mediante QR.

1.1.2. DETALLE DE CASOS DE USO

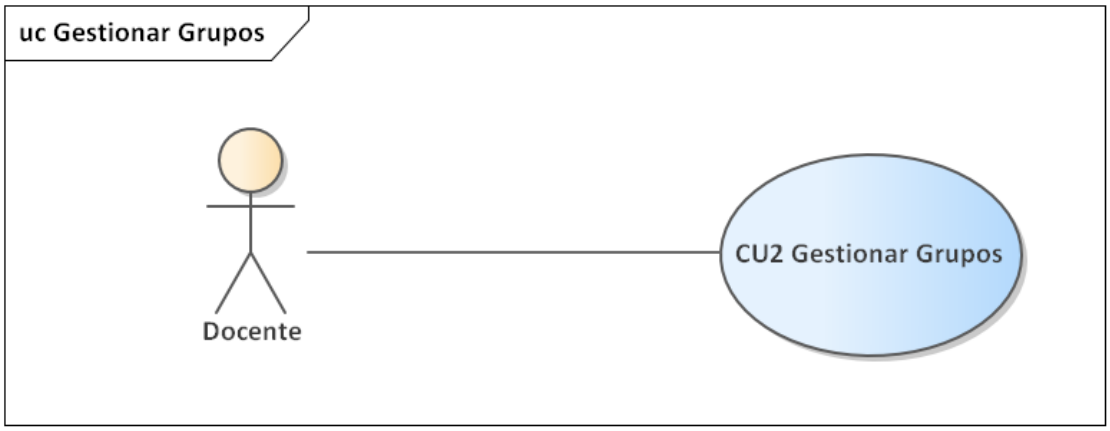
CU1: GESTIONAR MATERIAS



GESTIONAR MATERIAS	
Propósito	Gestionar materias
Descripción	Permite al docente gestionar las materias que imparte.
Actores	Docente
Actor Iniciador	Docente
Precondición	Ninguna
Proceso	**1. Crear materia**

	• Insertar datos. • Crear Materia. **2. Actualizar materia** • Identificar la materia. • Modificar datos. • Guardar datos. **3. Eliminar materia** • Identificar la materia. • Eliminar materia.
Postcondición	Se crea, actualiza o elimina el registro de materia.
Excepciones	La materia ya existe en el sistema.

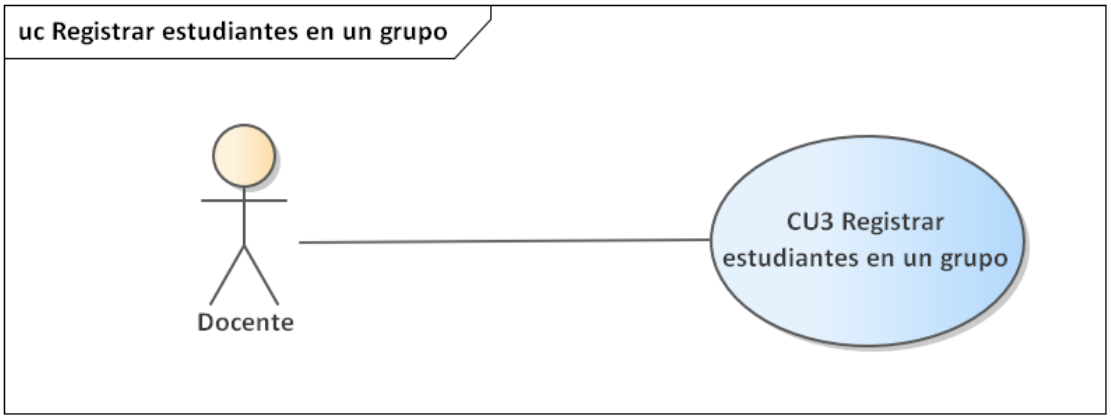
CU2: GESTIONAR GRUPOS



GESTIONAR GRUPOS	
Propósito	Gestionar grupos
Descripción	Permite al docente gestionar los grupos asociados a cada materia que imparte.
Actores	Docente
Actor Iniciador	Docente
Precondición	Tener al menos una materia registrada.
Proceso	**1. Crear grupo** • Insertar datos. • Seleccionar materia. • Crear grupo. **2. Actualizar grupo** • Identificar el grupo. • Modificar datos. • Guardar datos. **3. Eliminar grupo**

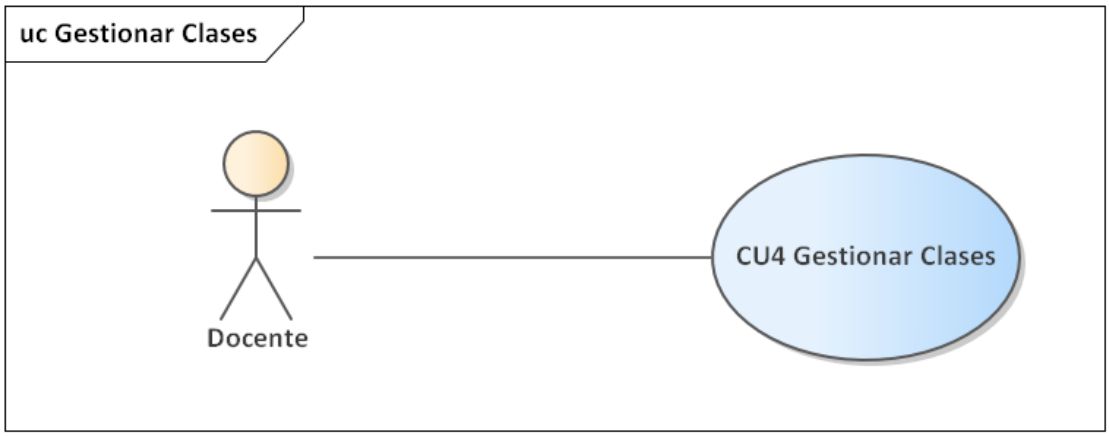
	• Identificar el grupo. • Eliminar grupo.
Postcondición	Se crea, actualiza o elimina el registro de grupo.
Excepciones	El grupo ya existe en la materia seleccionada.

CU3: REGISTRAR ESTUDIANTES EN UN GRUPO.



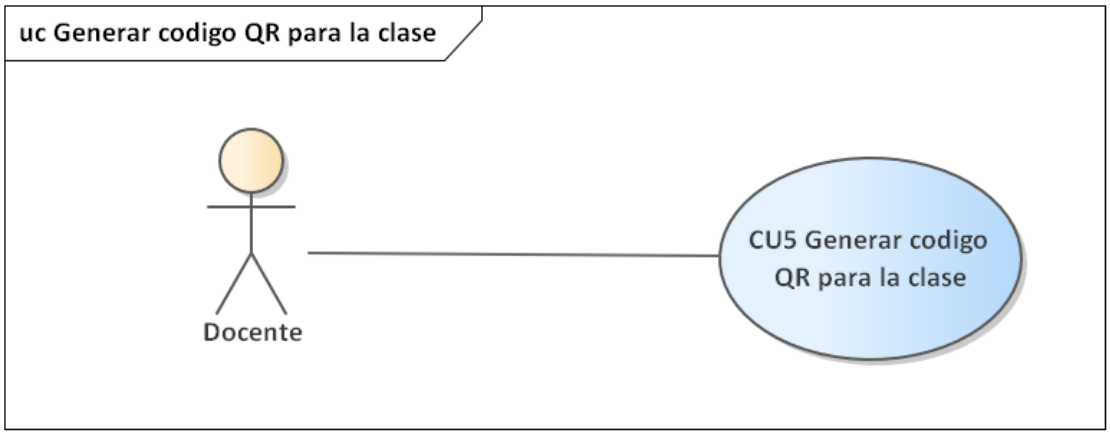
REGISTRAR ESTUDIANTES EN UN GRUPO.	
Propósito	Registrar estudiantes a un grupo
Descripción	Permite al docente registrar estudiantes en un grupo.
Actores	Docente
Actor Iniciador	Docente
Precondición	Tener al menos una materia y un grupo creados.
Proceso	**1. Registrar estudiantes a un grupo** • Identificar el grupo. • Seleccionar archivo Excel/CSV. • Mostrar la lista de estudiantes cargados. • Agregar o eliminar manualmente un estudiante, si es necesario. • Confirmar y guardar el registro de estudiantes en el grupo.
Postcondición	Los estudiantes quedan registrados y asociados al grupo seleccionado.
Excepciones	• El archivo no tiene un formato válido. • Existen datos incompletos o inválidos en el archivo. • El grupo no existe en el sistema. • Un estudiante ya se encuentra registrado en el grupo. • No existen estudiantes válidos para registrar.

CU4: GESTIONAR CLASES



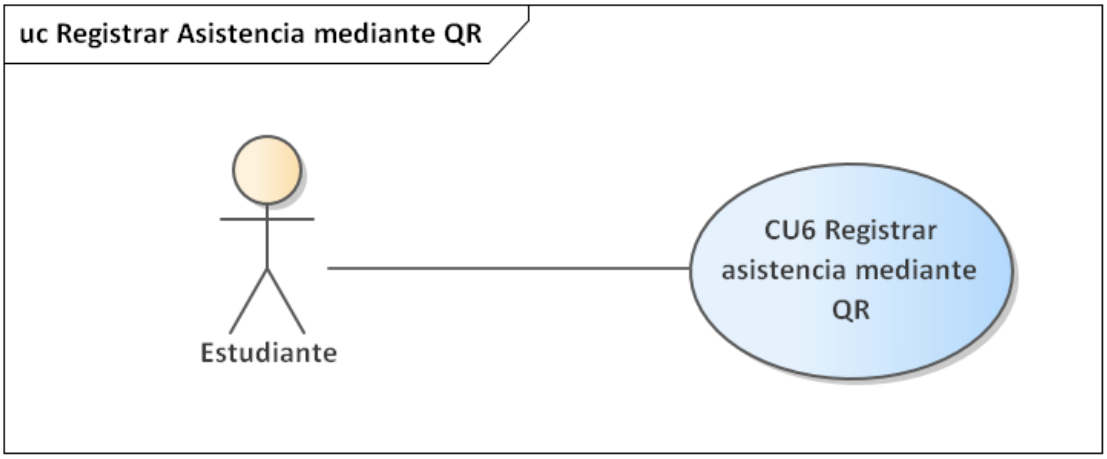
GESTIONAR CLASES	
Propósito	Gestionar clases.
Descripción	Permite al docente gestionar clases dentro de un grupo.
Actores	Docente
Actor Iniciador	Docente
Precondición	Tener al menos una materia y un grupo creados.
Proceso	1. **Crear clase** • Identificar el grupo. • Insertar datos. • Crear clase. 2. **Actualizar clase** • Identificar y seleccionar la clase. • Modificar datos. • Guardar cambios. 3. **Eliminar clase** • Identificar y seleccionar la clase. • Eliminar la clase.
Postcondición	Se crea, actualiza o elimina el registro de clase.
Excepciones	• El grupo no existe en el sistema. • La clase ya existe en el grupo seleccionado.

CU5: GENERAR CODIGO QR PARA LA CLASE



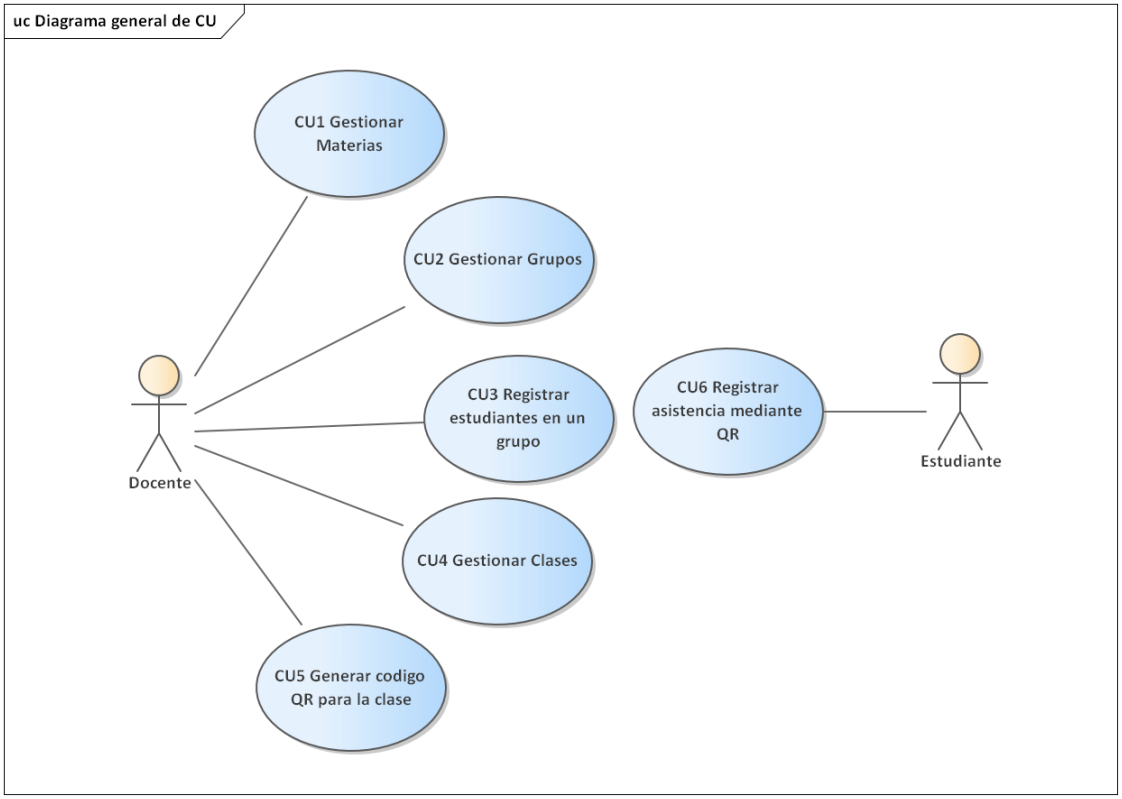
GENERAR CODIGO QR PARA LA CLASE	
Propósito	Generar codigo QR para la clase.
Descripción	Permite al docente generar un codigo QR para una clase programada, para que los estudiantes registren su asistencia.
Actores	Docente
Actor Iniciador	Docente
Precondición	Tener al menos una clase registrada.
Proceso	1. **Generar codigo QR para la clase** • Identificar la clase. • Validad fecha y horario de la clase. • Generar codigo QR. • Mostrar codigo QR.
Postcondición	El código QR queda generado y disponible para el registro de asistencia.
Excepciones	Ninguna.

CU6: REGISTRAR ASISTENCIA MEDIANTE QR



REGISTRAR ASISTENCIA MEDIANTE QR	
Propósito	Registrar asistencia mediante código QR.
Descripción	Permite al estudiante identificarse en el sistema y escanear un código QR para registrar su asistencia en la clase correspondiente.
Actores	Estudiante
Actor Iniciador	Estudiante
Precondición	Tener registro en el sistema.
Proceso	1. **Registrar asistencia mediante QR** • Identificarse en el sistema. • Escanear codigo QR. • Validad codigo QR. • Registrar asistencia.
Postcondición	La asistencia del estudiante queda registrada.
Excepciones	• El estudiante no existe en el sistema. • El código QR no es válido. • El código QR ha expirado. • La asistencia ya fue registrada anteriormente.

1.1.3. DIAGRAMA GENERAL DE CASOS DE USO



1.2. REQUISITOS NO FUNCIONALES

Usabilidad: La interfaz de usuario debe ser intuitiva y fácil de usar para el administrador.

Mantenibilidad: El código de la aplicación debe estar bien documentado y seguir buenas prácticas de desarrollo para facilitar futuras actualizaciones y mantenimiento.

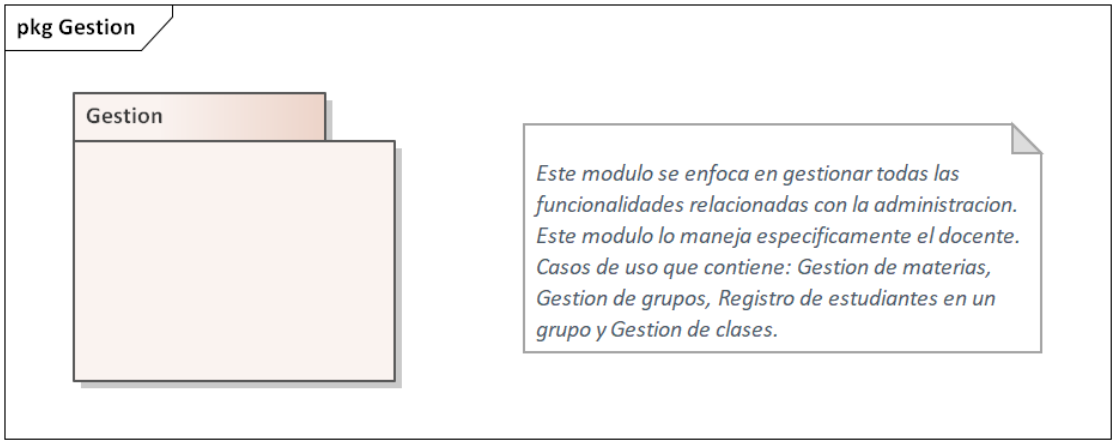
Escalabilidad: El sistema debe ser escalable para acomodar el crecimiento futuro de la cantidad de productos ofrecidos.

Compatibilidad: El software debe ser compatible con múltiples versiones del sistema operativo Android para garantizar la usabilidad.

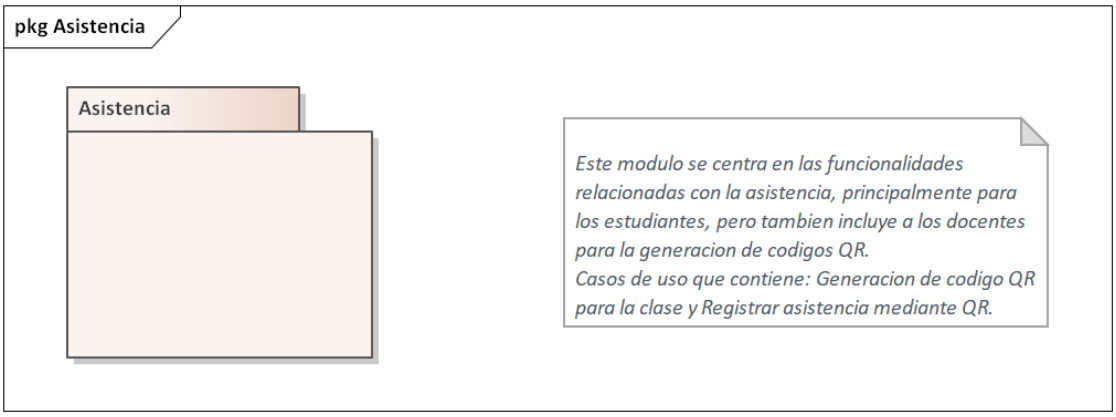
2. ANALISIS

2.1. IDENTIFICACION DE MODULO

Modulo 1: Gestión

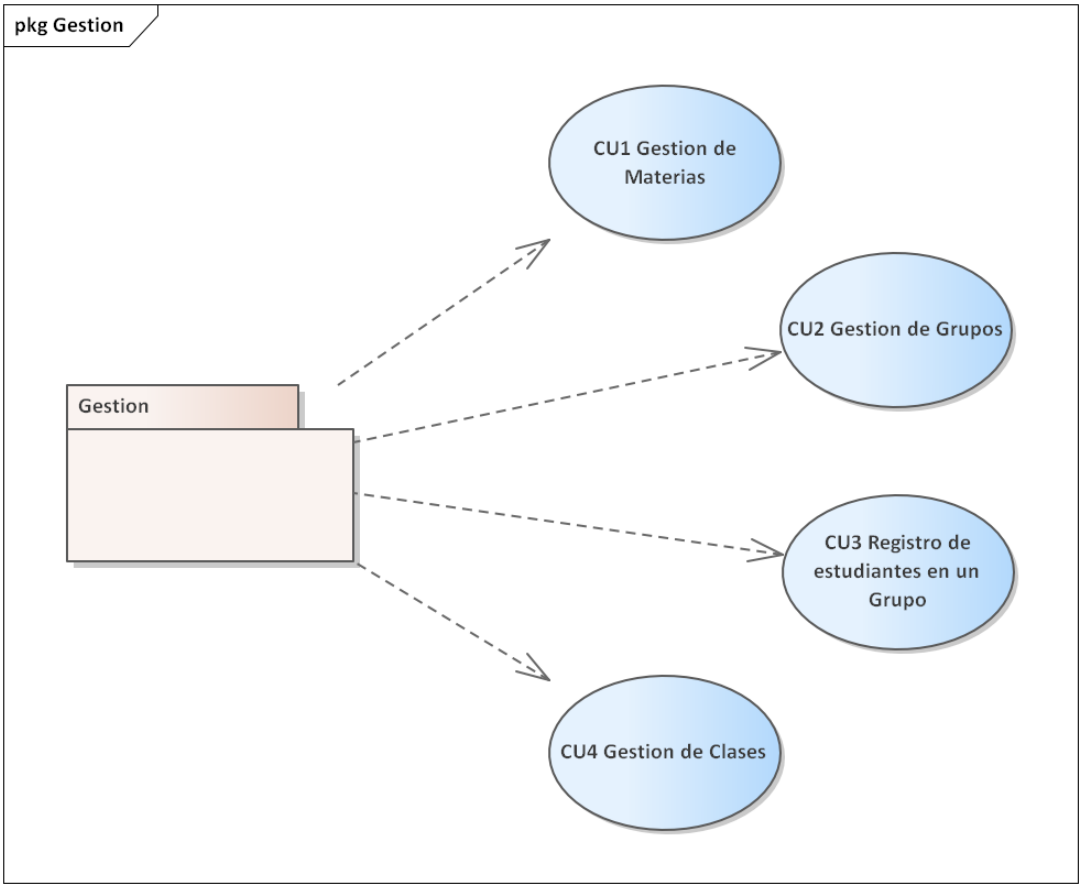


Modulo 2: Asistencia

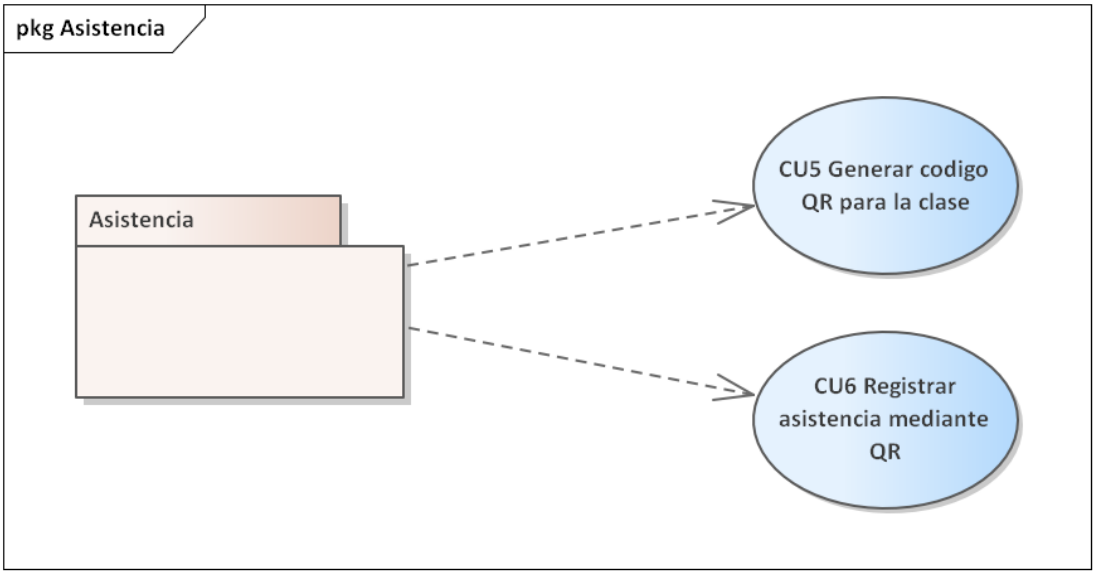


2.2. VISTA DE MODULOS

Modulo 1: Gestión

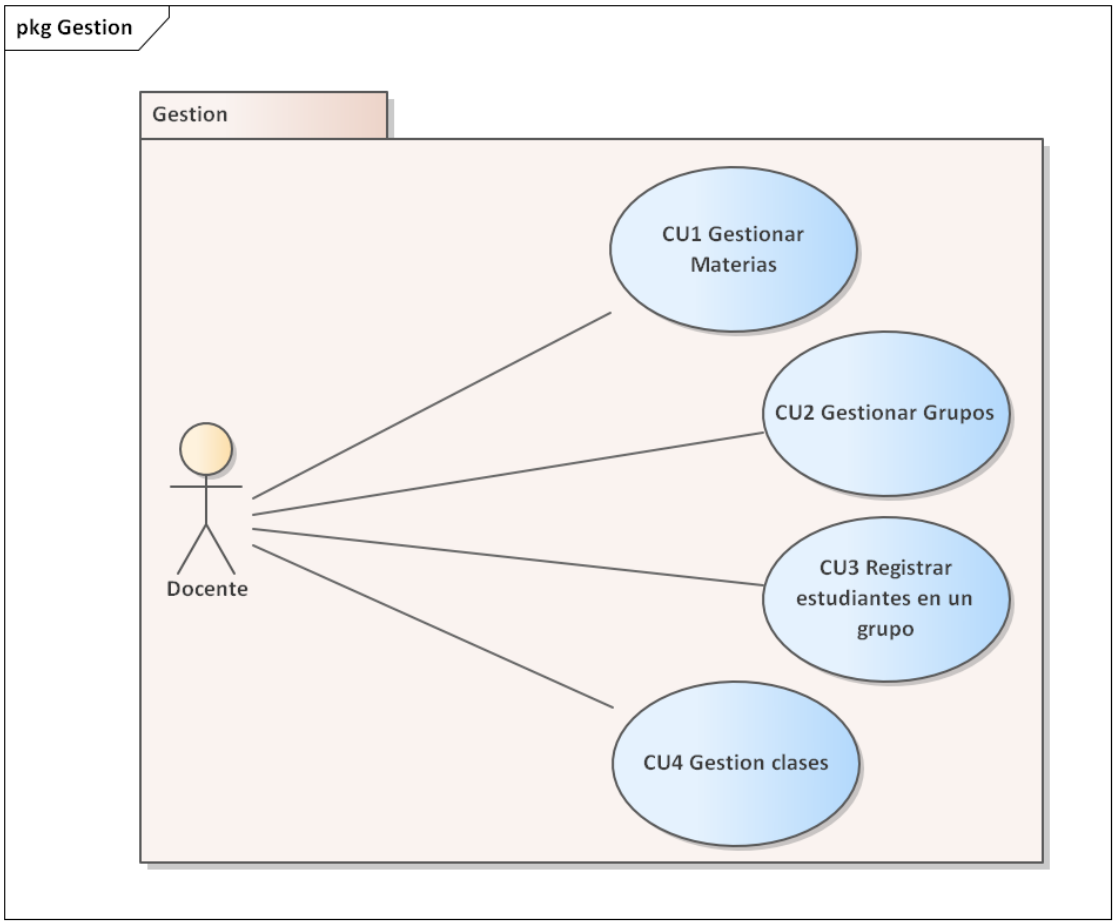


Modulo 2: Asistencia

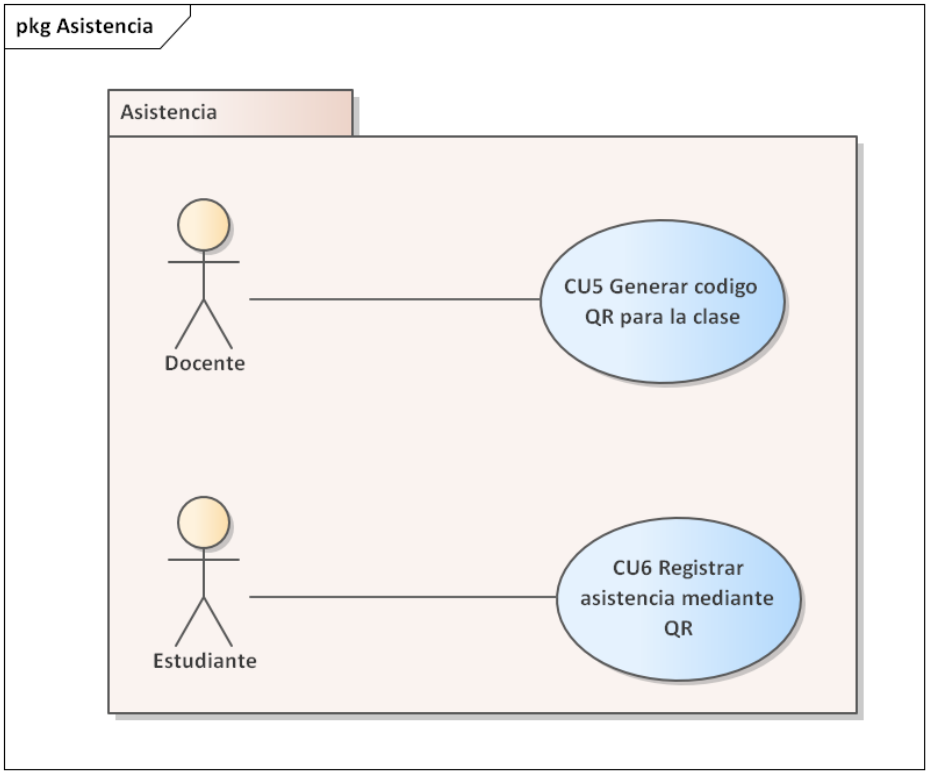


2.3. ENCAPSULAMIENTO DE CASOS DE USO

Modulo 1: Gestión



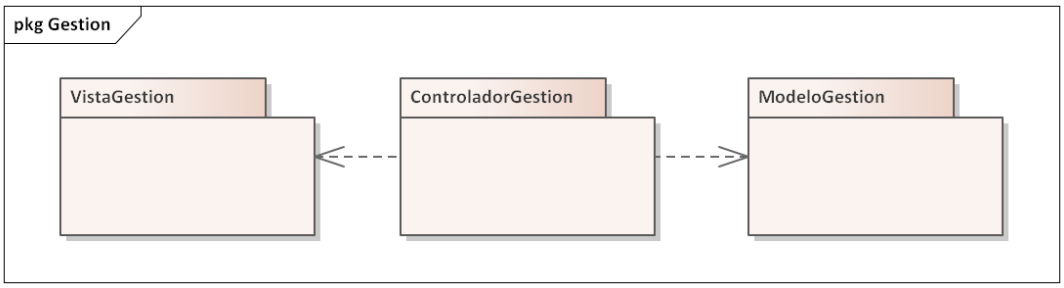
Modulo 2: Asistencia



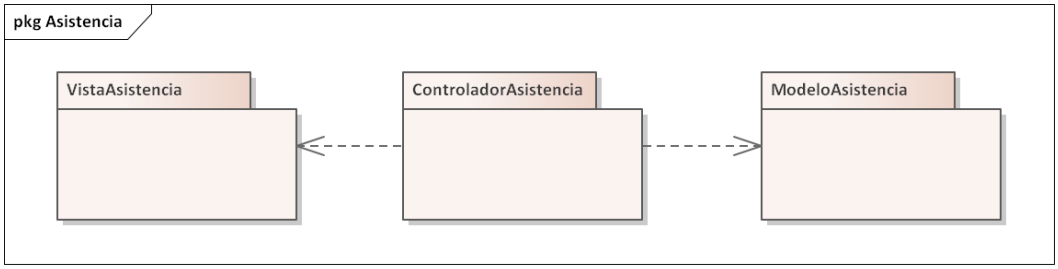
3. DISEÑO

3.1. DISEÑO DE LA ARQUITECTURA LOGICA

3.1.1. Módulo de Gestión

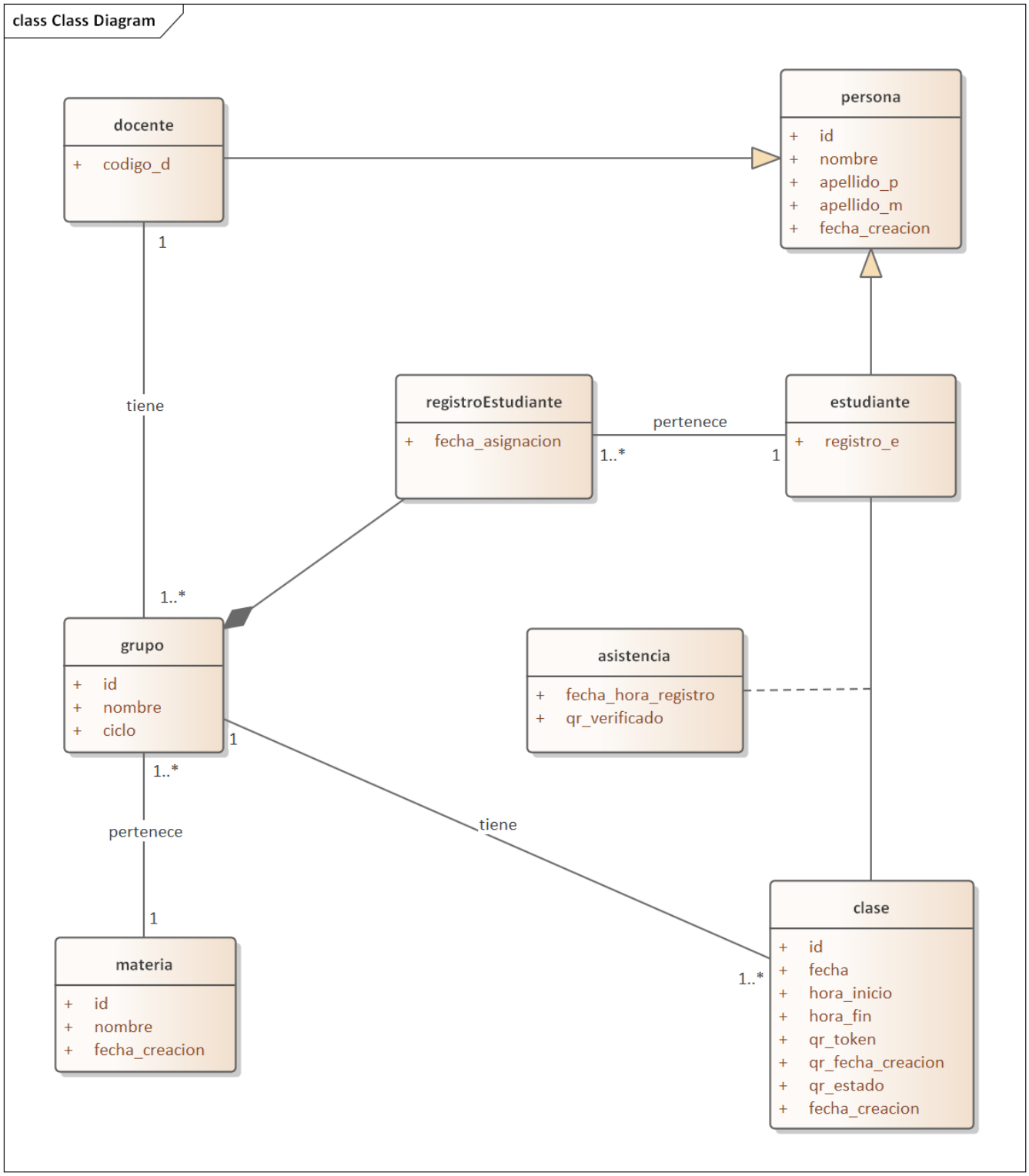


3.1.2. Módulo de Asistencia



3.2. DISEÑO DE LA BASE DE DATOS

3.2.1. DISEÑO CONCEPTUAL



3.2.2. DISEÑO LOGICO

Persona				
pk				
id	nombre	apellido_p	apellido_m	fecha_creacion

docente	
pk/fk	
persona_id	codigo_d

estudiante	
pk/fk	
persona_id	registro_e

materia		
Pk		
id	nombre	fecha_creacion

grupo					
pk			fk	fk	
id	nombre	ciclo	docente_id	materia_id	fecha_creacion

registroEstudiante			
pk		fk	fk
id	fecha_asignacion	estudiante_id	grupo_id

clase							
pk				fk			
id	fecha	hora_inicio	hora_fin	grupo_id	qr_token	qr-fecha_creacion	qr_estado

asistencia				
pk			fk	fk
id	fecha_hora_registro	qr_verificado	estudiante_id	clase_id

3.2.3. DISEÑO FISICO

```
-- PERSONA
CREATE TABLE persona (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
```

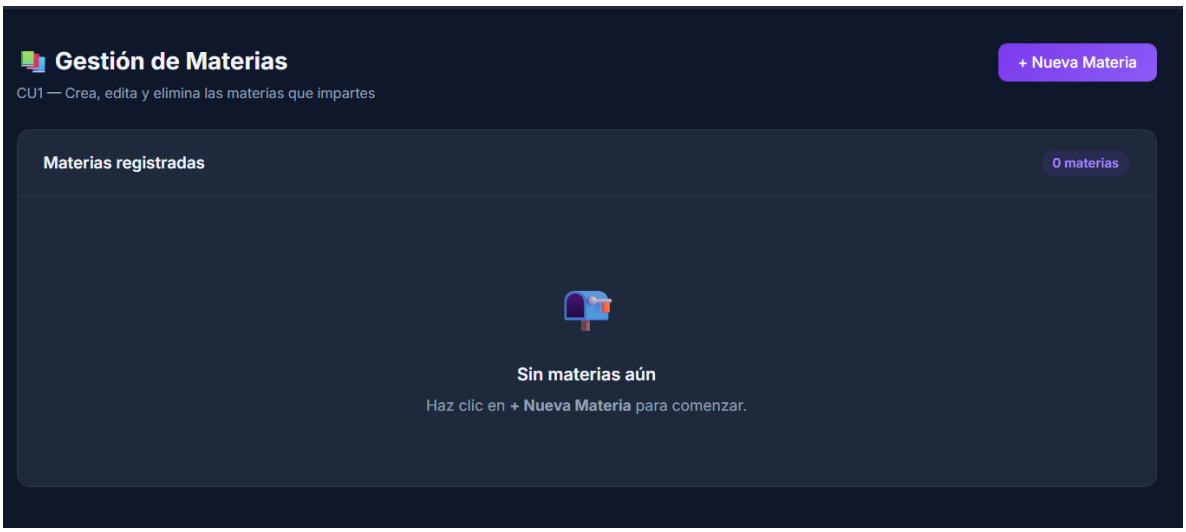
```
    apellido_p VARCHAR(100) NOT NULL,
    apellido_m VARCHAR(100),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- DOCENTE
CREATE TABLE docente (
    persona_id INT PRIMARY KEY,
    codigo_d INT UNIQUE NOT NULL,
    FOREIGN KEY (persona_id) REFERENCES persona(id) ON DELETE
CASCADE
);

-- ESTUDIANTE
CREATE TABLE estudiante (
    persona_id INT PRIMARY KEY,
    registro_e VARCHAR(50) UNIQUE NOT NULL,
    FOREIGN KEY (persona_id) REFERENCES persona(id) ON DELETE
CASCADE
);

-- MATERIA
CREATE TABLE materia (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- GRUPO
CREATE TABLE grupo (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    ciclo VARCHAR(50),
    docente_id INT NOT NULL,
    materia_id INT NOT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (docente_id) REFERENCES docente(persona_id) ON
DELETE CASCADE,
    FOREIGN KEY (materia_id) REFERENCES materia(id) ON DELETE
CASCADE,
```

```
    UNIQUE KEY uq_grupo (nombre, docente_id, materia_id)
);

-- REGISTRO ESTUDIANTE
CREATE TABLE registroEstudiante (
    id INT PRIMARY KEY AUTO_INCREMENT,
    fecha_asignacion DATE NOT NULL DEFAULT (CURRENT_DATE),
    estudiante_id INT NOT NULL,
    grupo_id INT NOT NULL,
    FOREIGN KEY (estudiante_id) REFERENCES estudiante(persona_id)
ON DELETE CASCADE,
    FOREIGN KEY (grupo_id) REFERENCES grupo(id) ON DELETE CASCADE,
    UNIQUE KEY uq_registro (estudiante_id, grupo_id)
);

-- CLASE
CREATE TABLE clase (
    id INT PRIMARY KEY AUTO_INCREMENT,
    fecha DATE NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    grupo_id INT NOT NULL,
    qr_token VARCHAR(255) UNIQUE NULL COMMENT 'Token único para el
código QR vigente',
    qr_fecha_creacion DATETIME NULL COMMENT 'Fecha y hora en que
se generó el QR',
    qr_estado ENUM('vigente', 'expirado', 'invalidado') DEFAULT
'vigente' COMMENT 'Estado del QR actual',
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (grupo_id) REFERENCES grupo(id) ON DELETE CASCADE
);

-- ASISTENCIA
CREATE TABLE asistencia (
    id INT PRIMARY KEY AUTO_INCREMENT,
    fecha_hora_registro DATETIME NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    qr_verificado BOOLEAN DEFAULT TRUE,
    estudiante_id INT NOT NULL,
    clase_id INT NOT NULL,
```

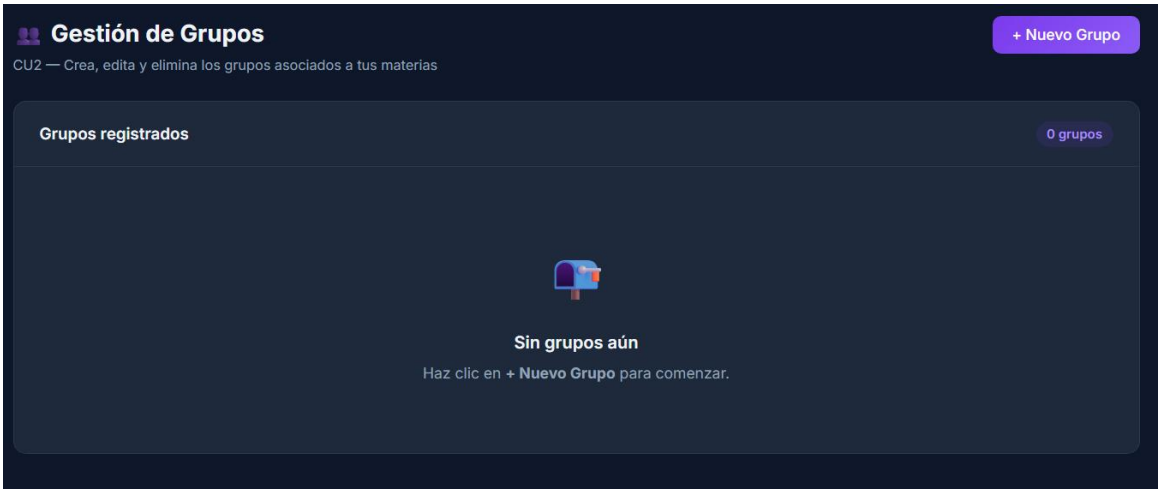
```
    FOREIGN KEY (estudiante_id) REFERENCES estudiante(persona_id)
ON DELETE CASCADE,
    FOREIGN KEY (clase_id) REFERENCES clase(id) ON DELETE CASCADE,
    UNIQUE KEY uq_asistencia (estudiante_id, clase_id)
);
```

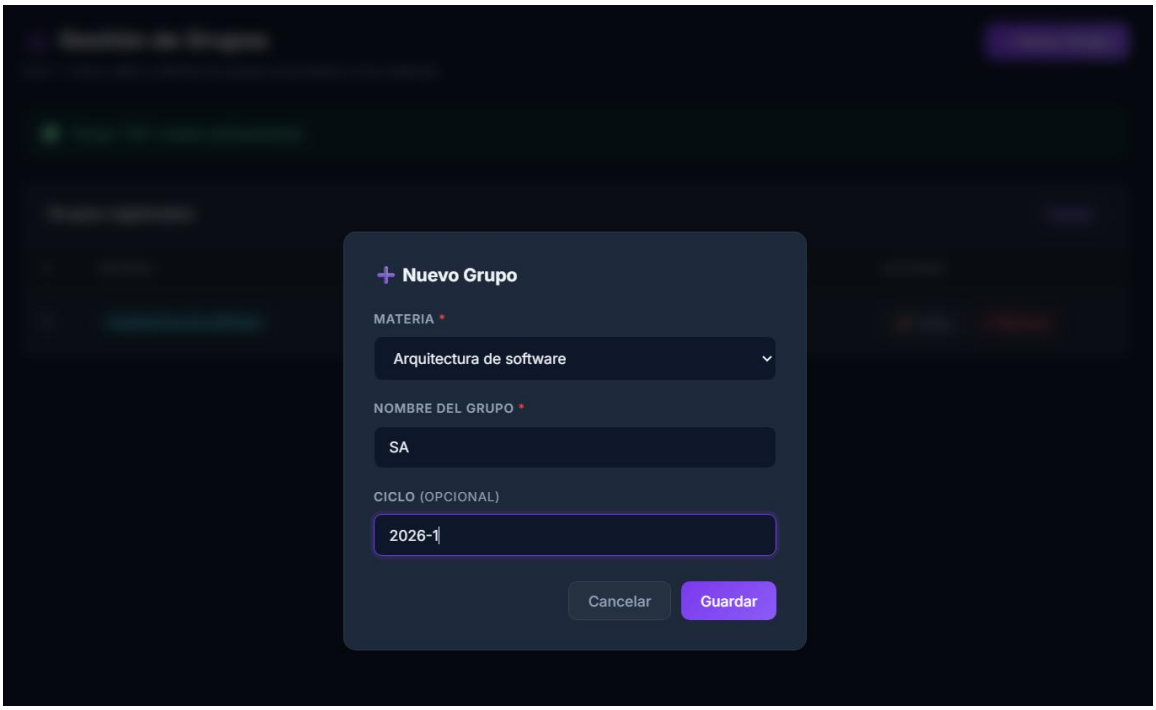
3.3. DISEÑO DE LA INTERFAZ

3.3.1. CU1 Gestión de Materias

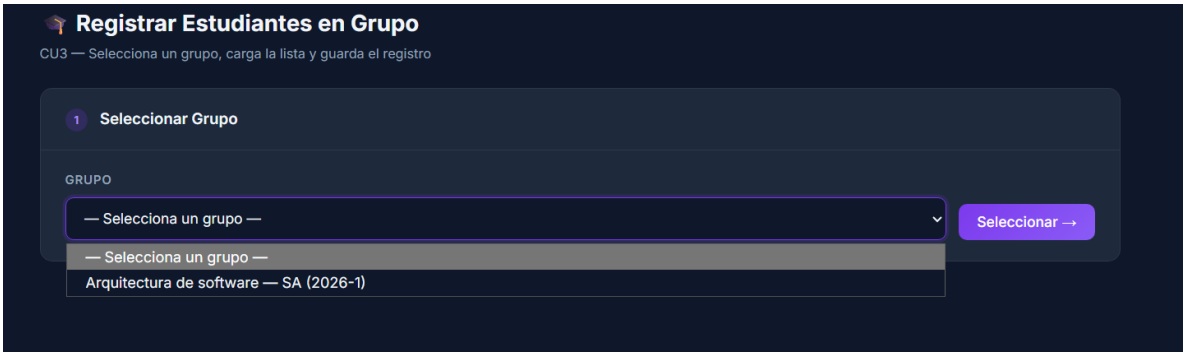


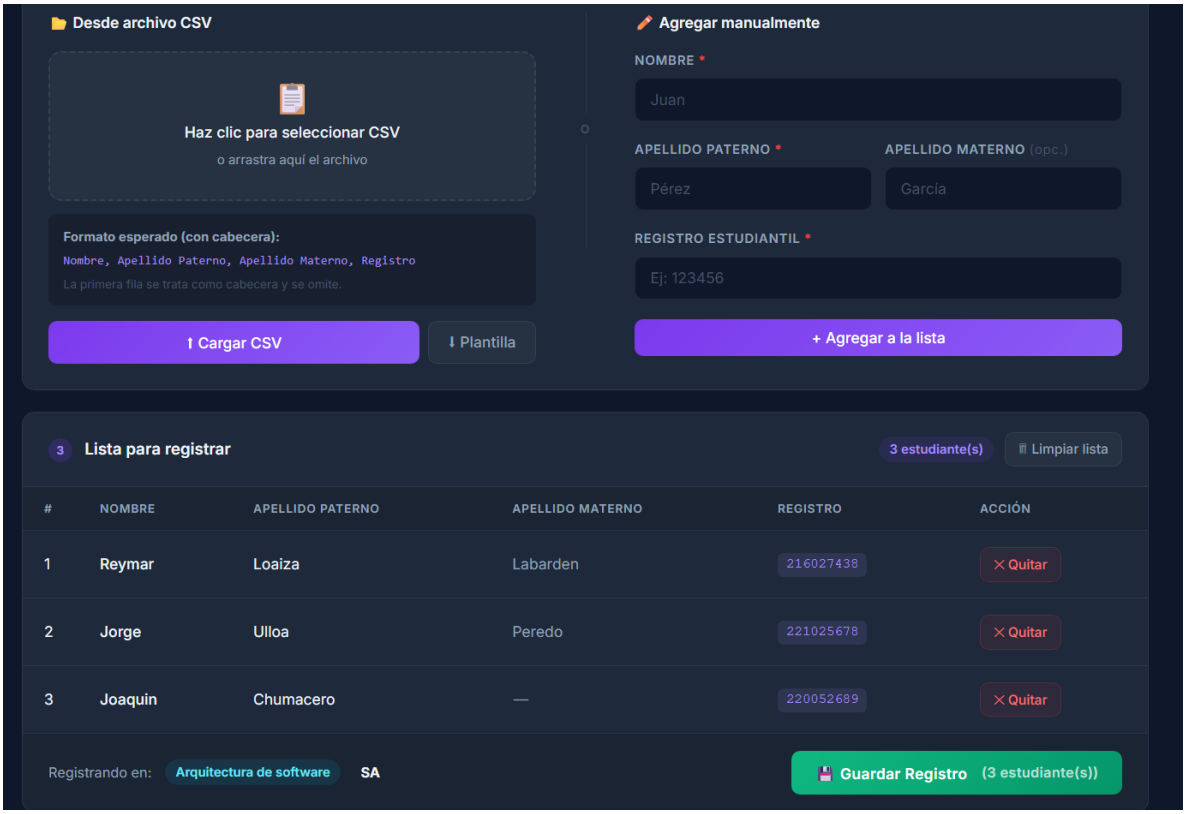
3.3.2. CU2 Gestión de Grupos



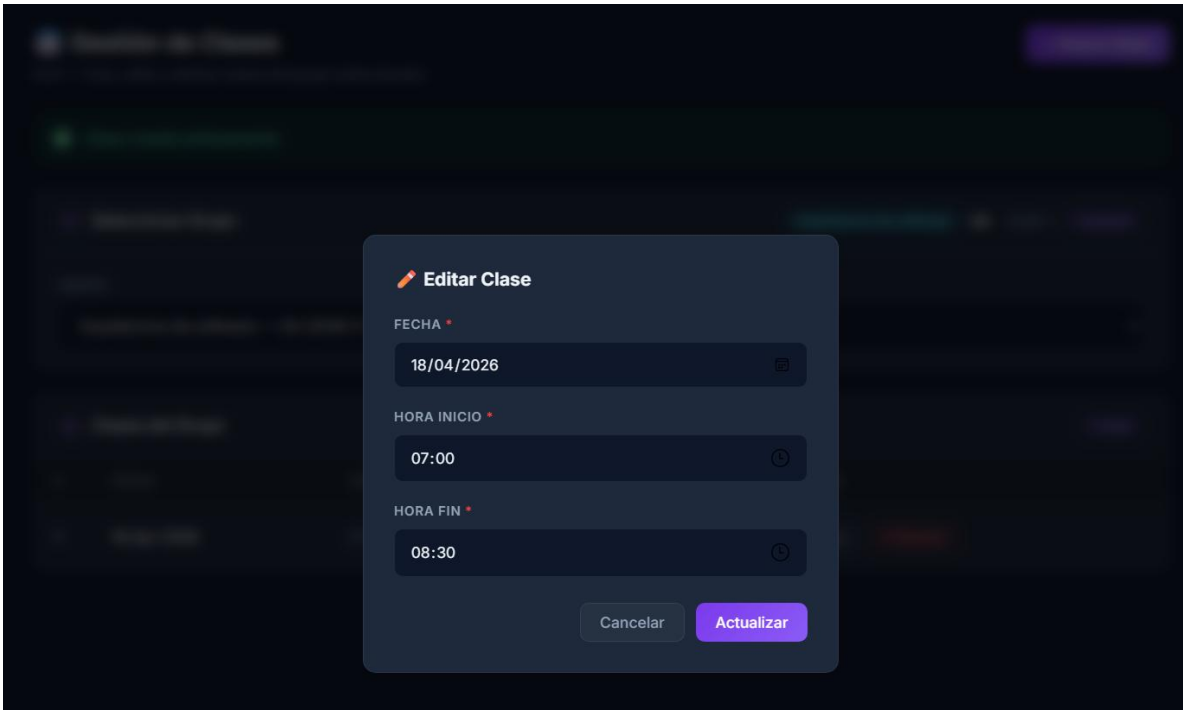


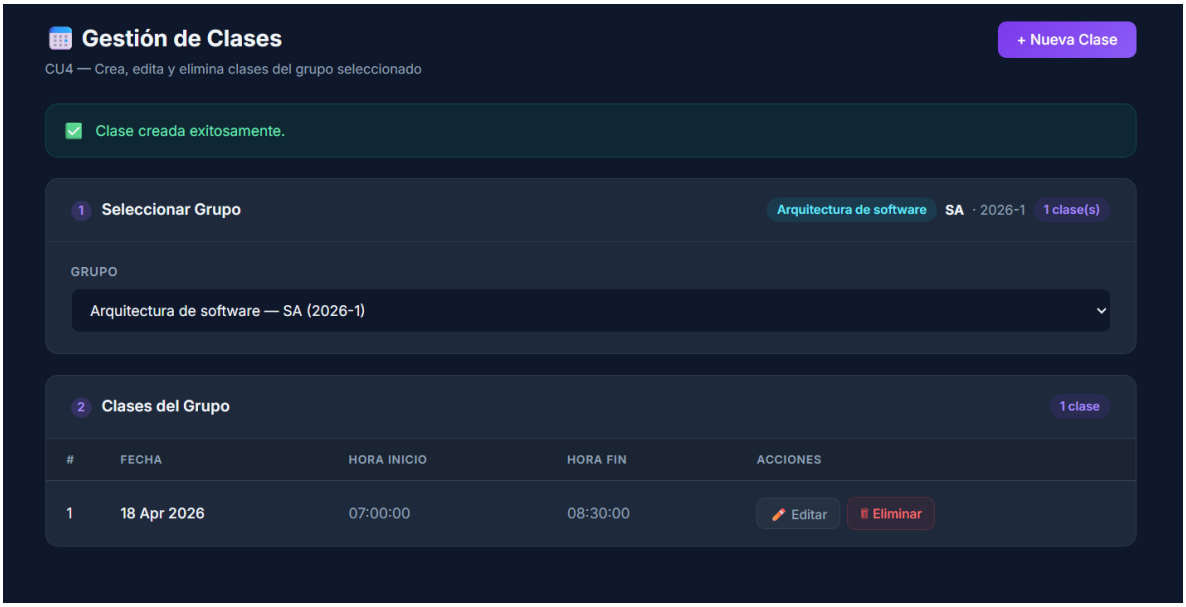
3.3.3. CU3 Registrar estudiantes en un Grupo.



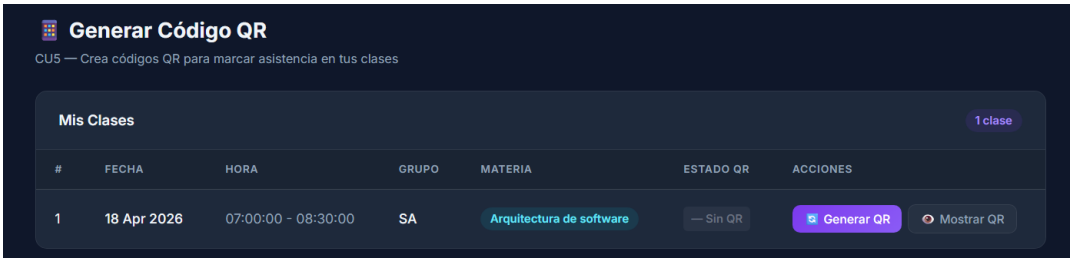


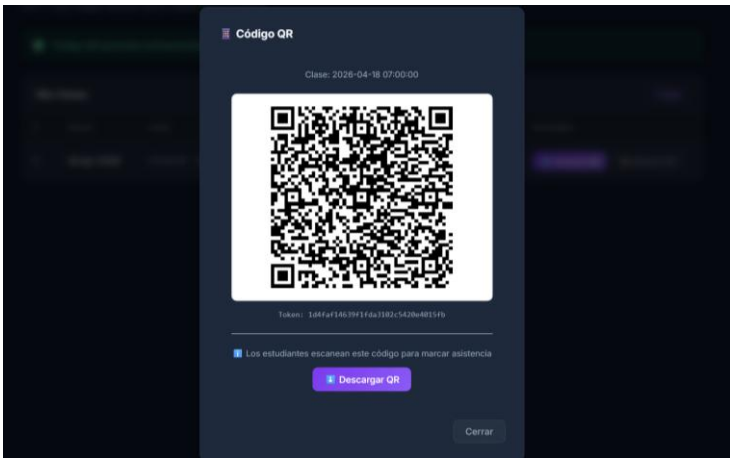
3.3.4. CU4 Gestionar Clases.



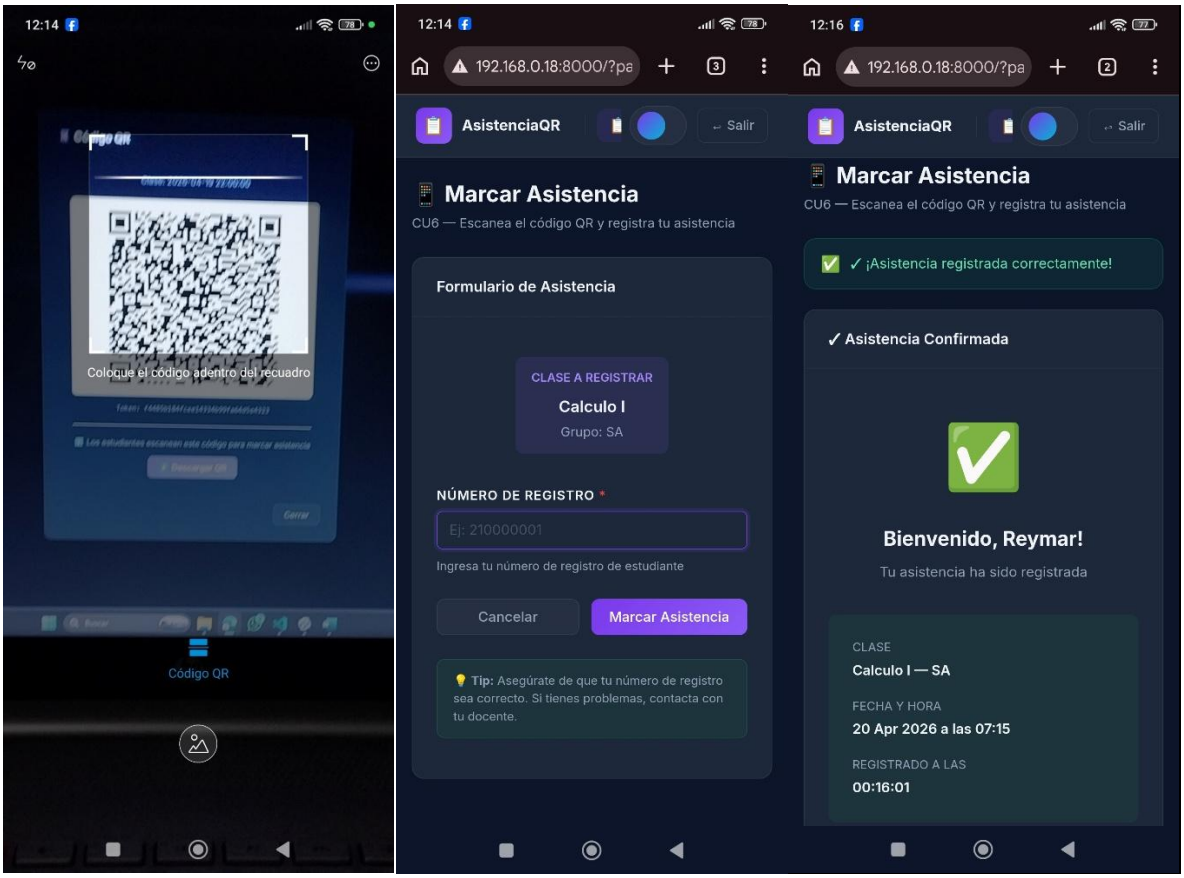


3.3.5. CU5 Generar código QR para la clase.



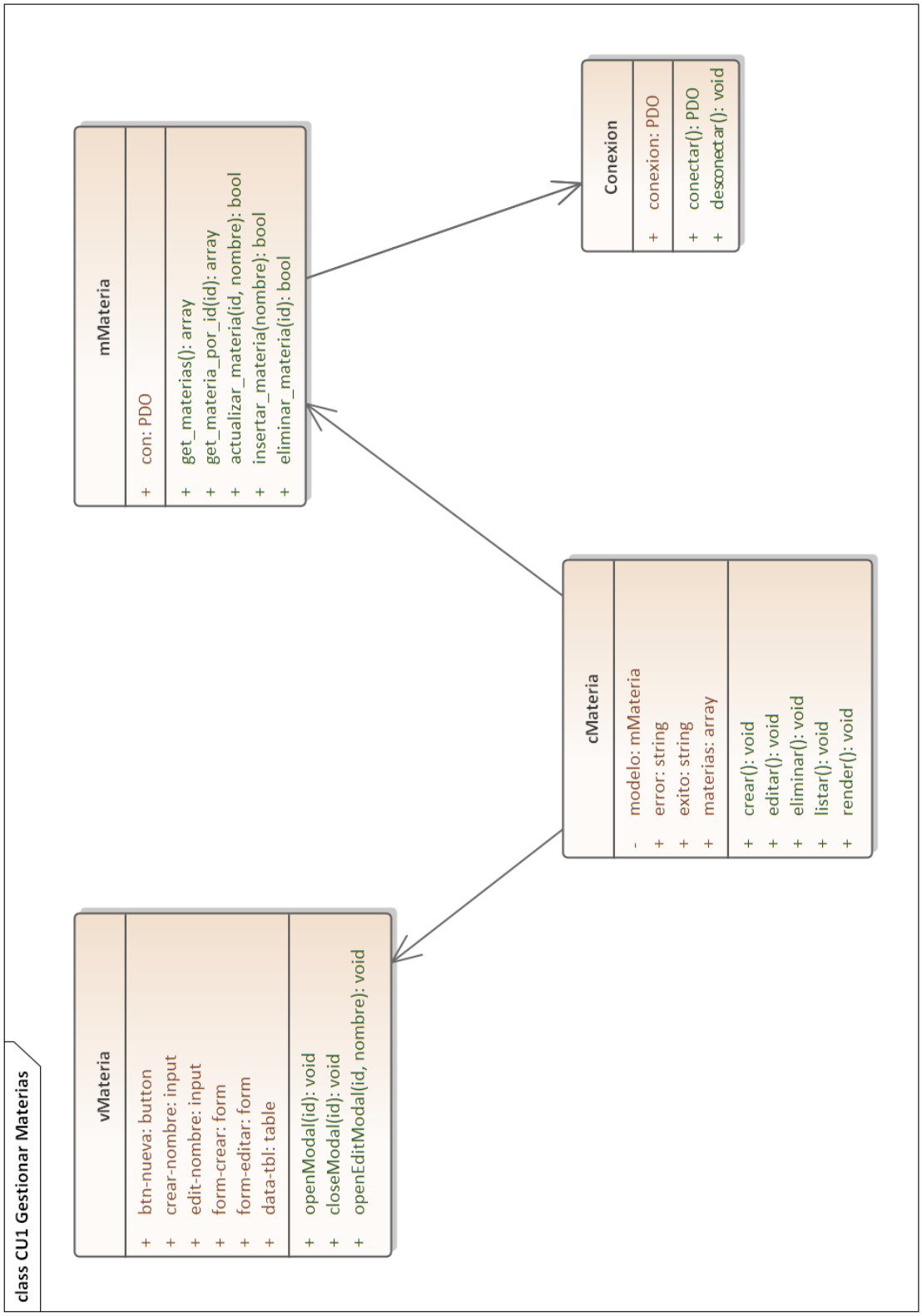


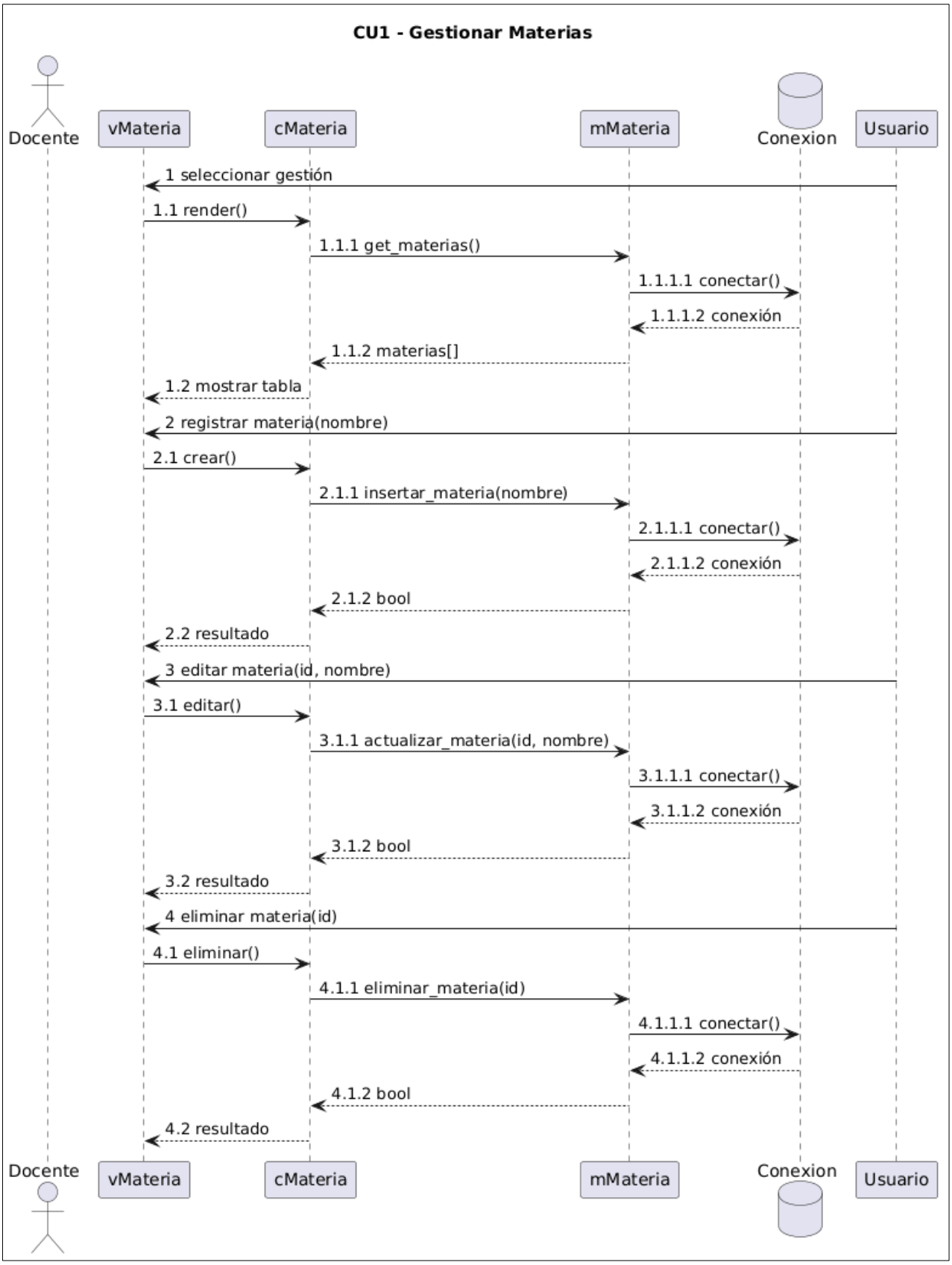
3.3.6. CU6 Registrar asistencia mediante QR.



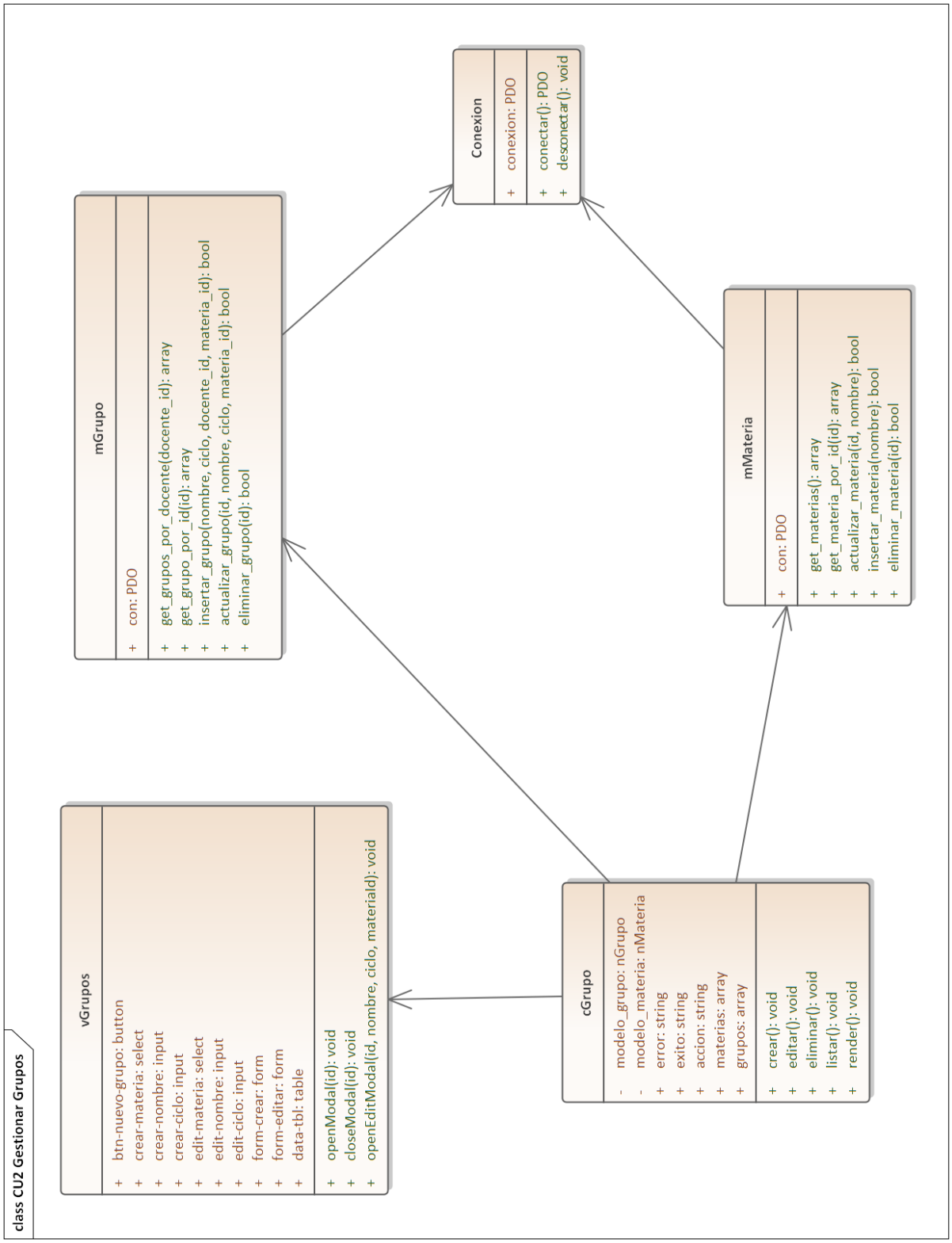
3.4. DISEÑO DE DETALLE PROCEDIMENTAL

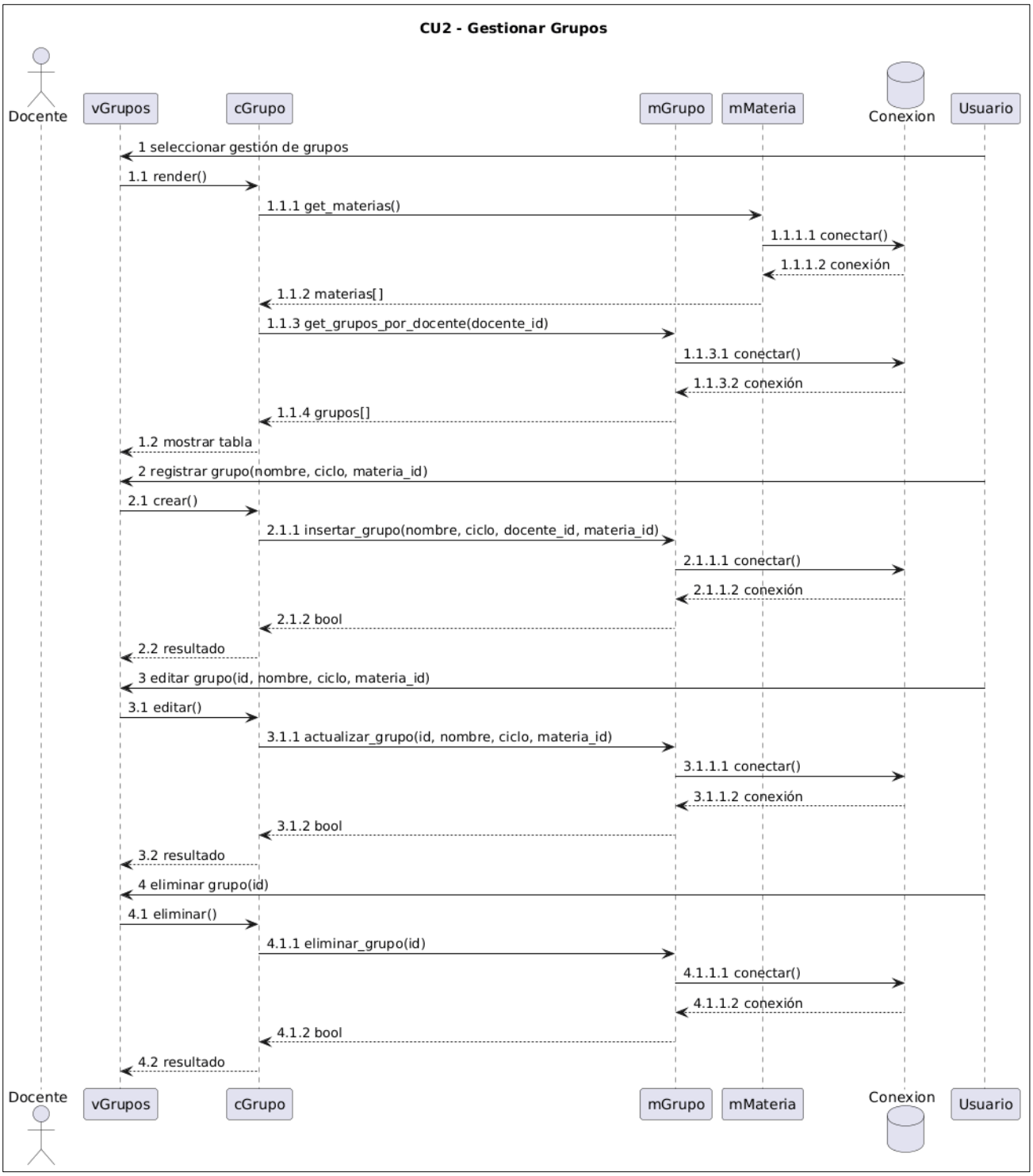
3.4.1. CU1 Gestión de Materias



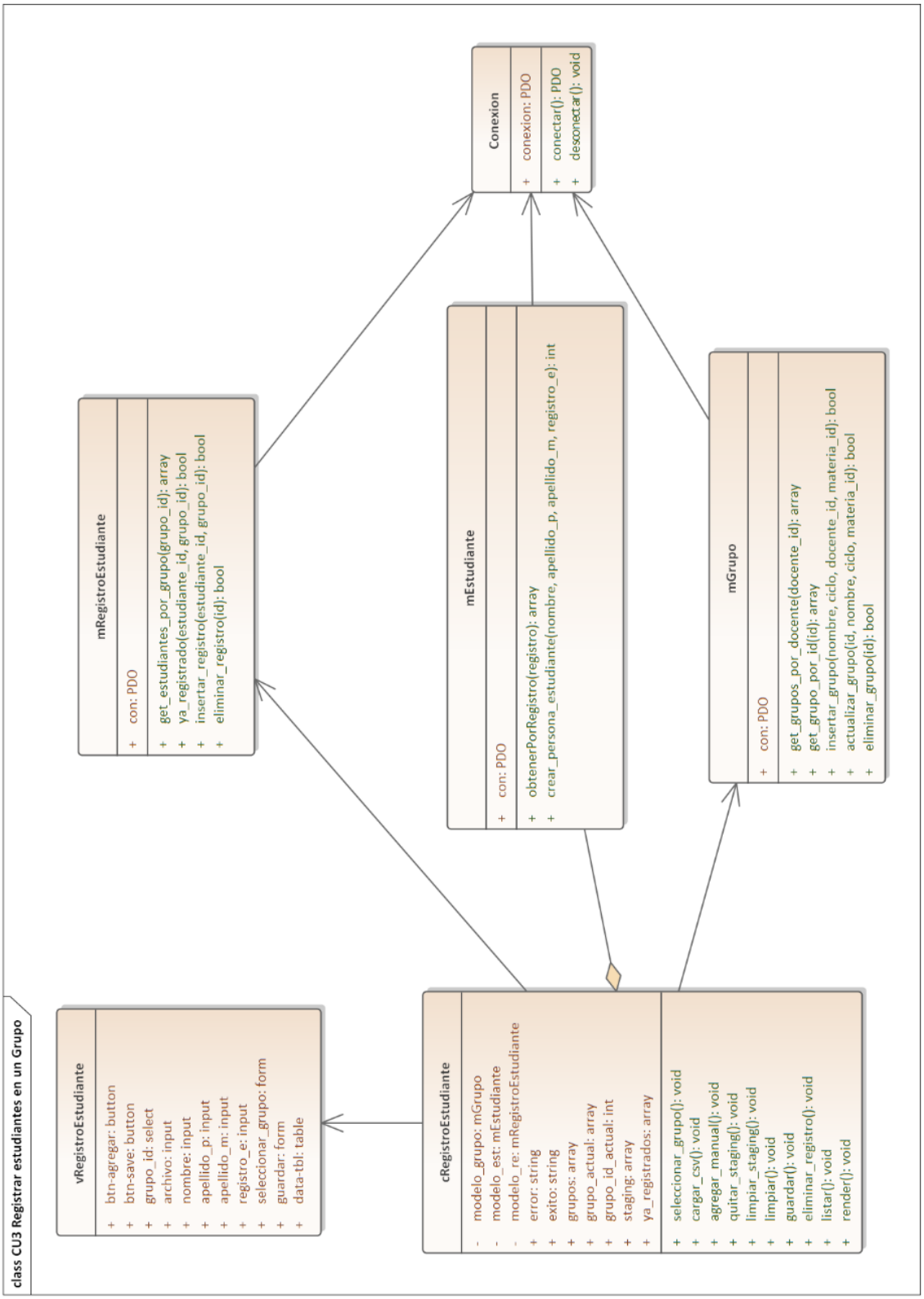

CU1 - Gestionar Materias
Docente
vMateria
cMateria
mMateria
Conexion
Usuario
1 seleccionar gestión
1.1 render()
1.1.1 get_materias()
1.1.1.1 conectar()
1.1.1.2 conexión
1.1.2 materias[]
1.2 mostrar tabla
2 registrar materia(nombre)
2.1 crear()
2.1.1 insertar_materia(nombre)
2.1.1.1 conectar()
2.1.1.2 conexión
2.1.2 bool
2.2 resultado
3 editar materia(id, nombre)
3.1 editar()
3.1.1 actualizar_materia(id, nombre)
3.1.1.1 conectar()
3.1.1.2 conexión
3.1.2 bool
3.2 resultado
4 eliminar materia(id)
4.1 eliminar()
4.1.1 eliminar_materia(id)
4.1.1.1 conectar()
4.1.1.2 conexión
4.1.2 bool
4.2 resultado
Docente
vMateria
cMateria
mMateria
Conexion
Usuario

3.4.2. CU2 Gestión de Grupos



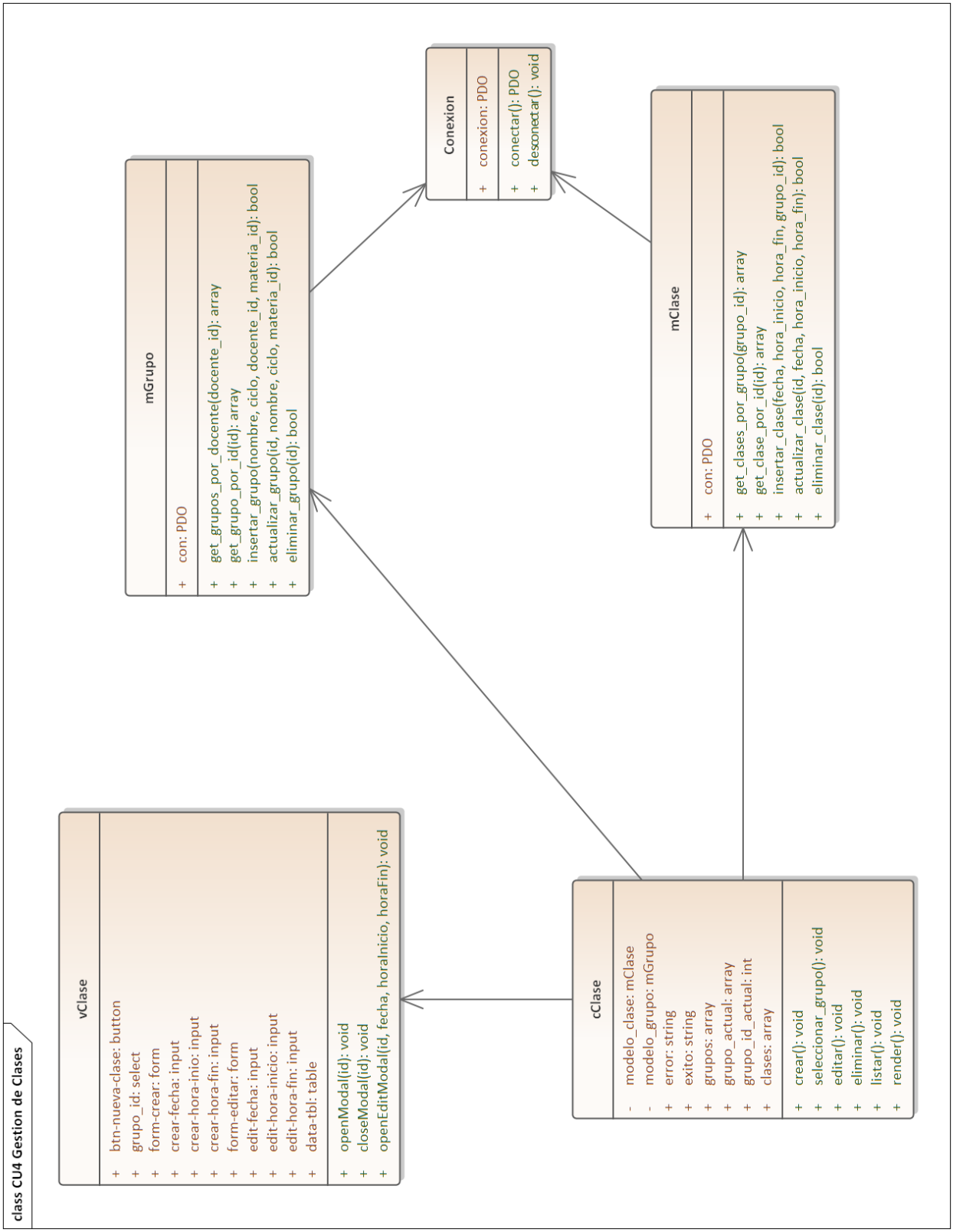

CU2 - Gestionar Grupos
Docente
vGrupos
cGrupo
mGrupo
mMateria
Conexion
Usuario
1 seleccionar gestión de grupos
1.1 render()
1.1.1 get_materias()
1.1.1.1 conectar()
1.1.1.2 conexión
1.1.2 materias[]
1.1.3 get_grupos_por_docente(docente_id)
1.1.3.1 conectar()
1.1.3.2 conexión
1.1.4 grupos[]
1.2 mostrar tabla
2 registrar grupo(nombre, ciclo, materia_id)
2.1 crear()
2.1.1 insertar_grupo(nombre, ciclo, docente_id, materia_id)
2.1.1.1 conectar()
2.1.1.2 conexión
2.1.2 bool
2.2 resultado
3 editar grupo(id, nombre, ciclo, materia_id)
3.1 editar()
3.1.1 actualizar_grupo(id, nombre, ciclo, materia_id)
3.1.1.1 conectar()
3.1.1.2 conexión
3.1.2 bool
3.2 resultado
4 eliminar grupo(id)
4.1 eliminar()
4.1.1 eliminar_grupo(id)
4.1.1.1 conectar()
4.1.1.2 conexión
4.1.2 bool
4.2 resultado

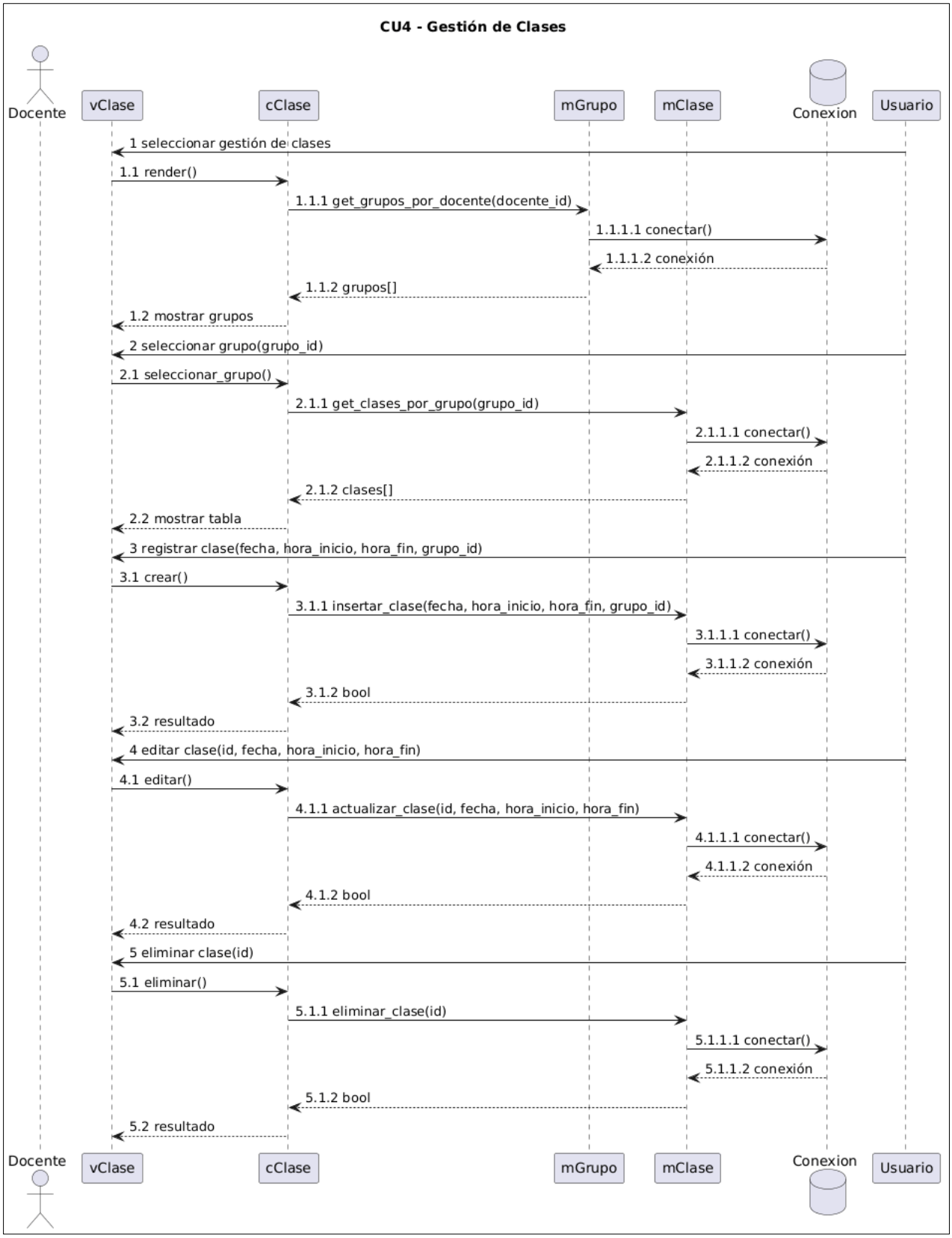
3.4.3. CU3 Registrar estudiantes en un Grupo



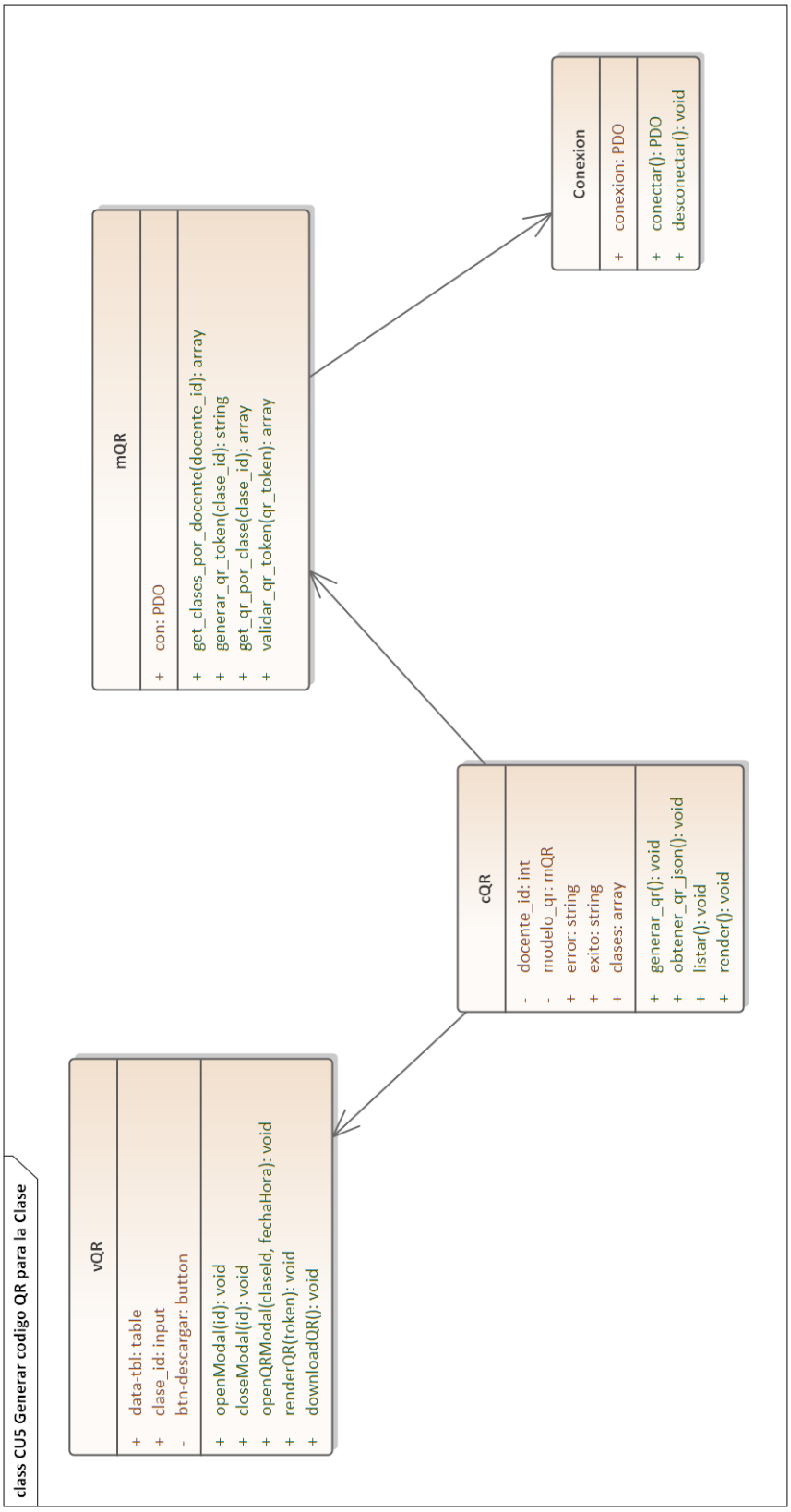
CU3 - Registrar estudiantes en un Grupo
Docente
vRegistroEstudiante
cRegistroEstudiante
mGrupo
mEstudiante
mRegistroEstudiante
Conexion
Usuario
1 seleccionar registro de estudiantes
1.1 render()
1.1.1 get_grupos_por_docente(docente_id)
1.1.1.1 conectar()
1.1.1.2 conexión
1.1.2 grupos[]
1.2 mostrar grupos
2 seleccionar grupo(grupo_id)
2.1 seleccionar_grupo()
2.1.1 get_estudiantes_por_grupo(grupo_id)
2.1.1.1 conectar()
2.1.1.2 conexión
2.1.2 ya_registrados[]
2.2 mostrar estudiantes del grupo
3 buscar estudiante(registro)
3.1 agregar_manual()
3.1.1 obtenerPorRegistro(registro)
3.1.1.1 conectar()
3.1.1.2 conexión
3.1.2 estudiante[]
3.2 mostrar estudiante
4 confirmar agregar estudiante
4.1 guardar()
4.1.1 ya_registrado(estudiante_id, grupo_id)
4.1.1.1 conectar()
4.1.1.2 conexión
4.1.2 bool
4.1.3 insertar_registro(estudiante_id, grupo_id)
4.1.3.1 conectar()
4.1.3.2 conexión
4.1.4 bool
4.2 resultado
5 registrar estudiante nuevo
5.1 guardar()
5.1.1 crear_persona_estudiante(nombre, apellido_p, apellido_m, registro_e)
5.1.1.1 conectar()
5.1.1.2 conexión
5.1.2 estudiante_id
5.1.3 insertar_registro(estudiante_id, grupo_id)
5.1.3.1 conectar()
5.1.3.2 conexión
5.1.4 bool
5.2 resultado
6 eliminar registro(estudiante_id, grupo_id)
6.1 eliminar_registro()
6.1.1 eliminar_registro(id)
6.1.1.1 conectar()
6.1.1.2 conexión
6.1.2 bool
6.2 resultado

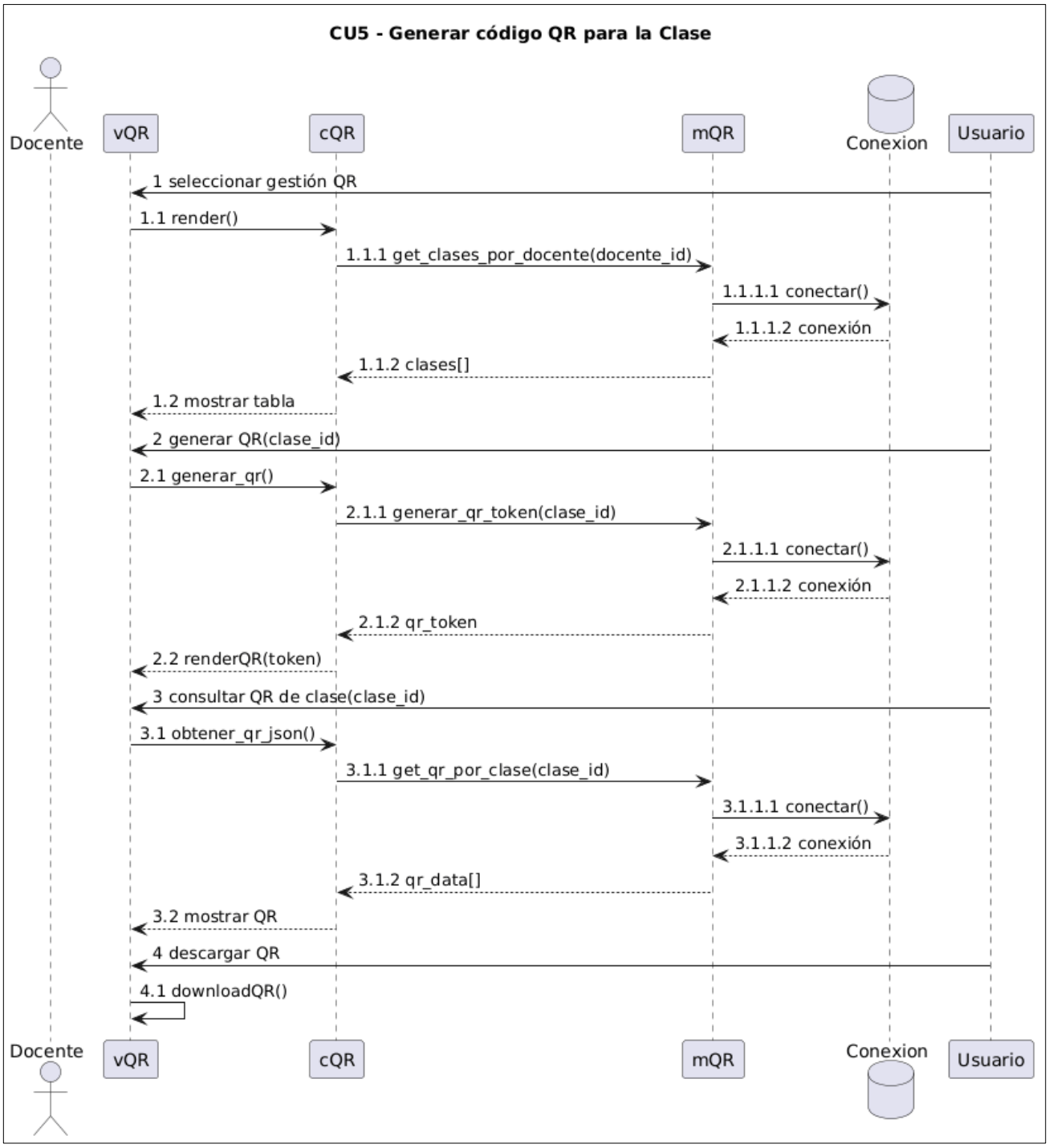
3.4.4. CU4 Gestión de Clases



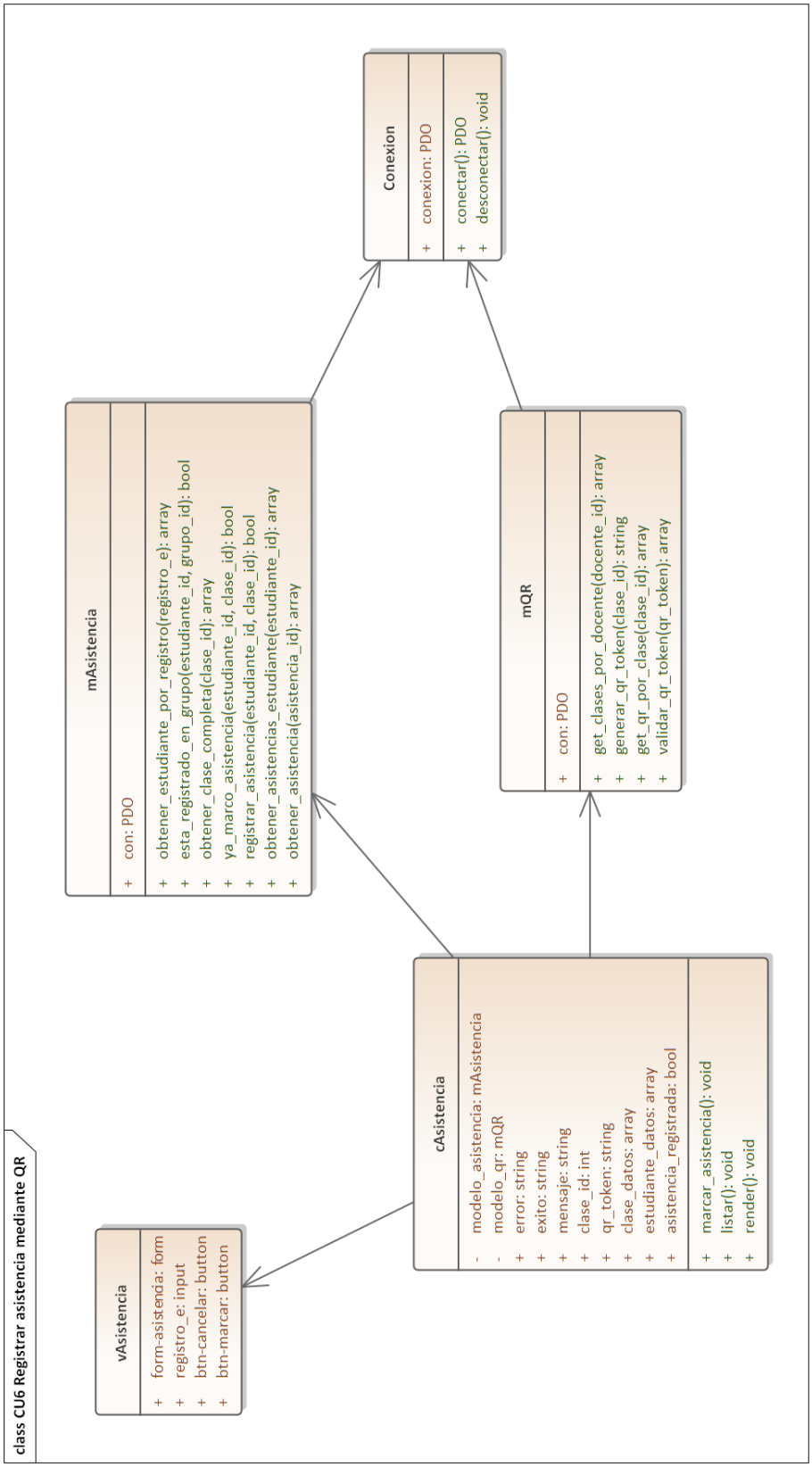

CU4 - Gestión de Clases
Docente
vClase
cClase
mGrupo
mClase
Conexion
Usuario
1 seleccionar gestión de clases
1.1 render()
1.1.1 get_grupos_por_docente(docente_id)
1.1.1.1 conectar()
1.1.1.2 conexión
1.1.2 grupos[]
1.2 mostrar grupos
2 seleccionar grupo(grupo_id)
2.1 seleccionar_grupo()
2.1.1 get_clases_por_grupo(grupo_id)
2.1.1.1 conectar()
2.1.1.2 conexión
2.1.2 clases[]
2.2 mostrar tabla
3 registrar clase(fecha, hora_inicio, hora_fin, grupo_id)
3.1 crear()
3.1.1 insertar_clase(fecha, hora_inicio, hora_fin, grupo_id)
3.1.1.1 conectar()
3.1.1.2 conexión
3.1.2 bool
3.2 resultado
4 editar clase(id, fecha, hora_inicio, hora_fin)
4.1 editar()
4.1.1 actualizar_clase(id, fecha, hora_inicio, hora_fin)
4.1.1.1 conectar()
4.1.1.2 conexión
4.1.2 bool
4.2 resultado
5 eliminar clase(id)
5.1 eliminar()
5.1.1 eliminar_clase(id)
5.1.1.1 conectar()
5.1.1.2 conexión
5.1.2 bool
5.2 resultado

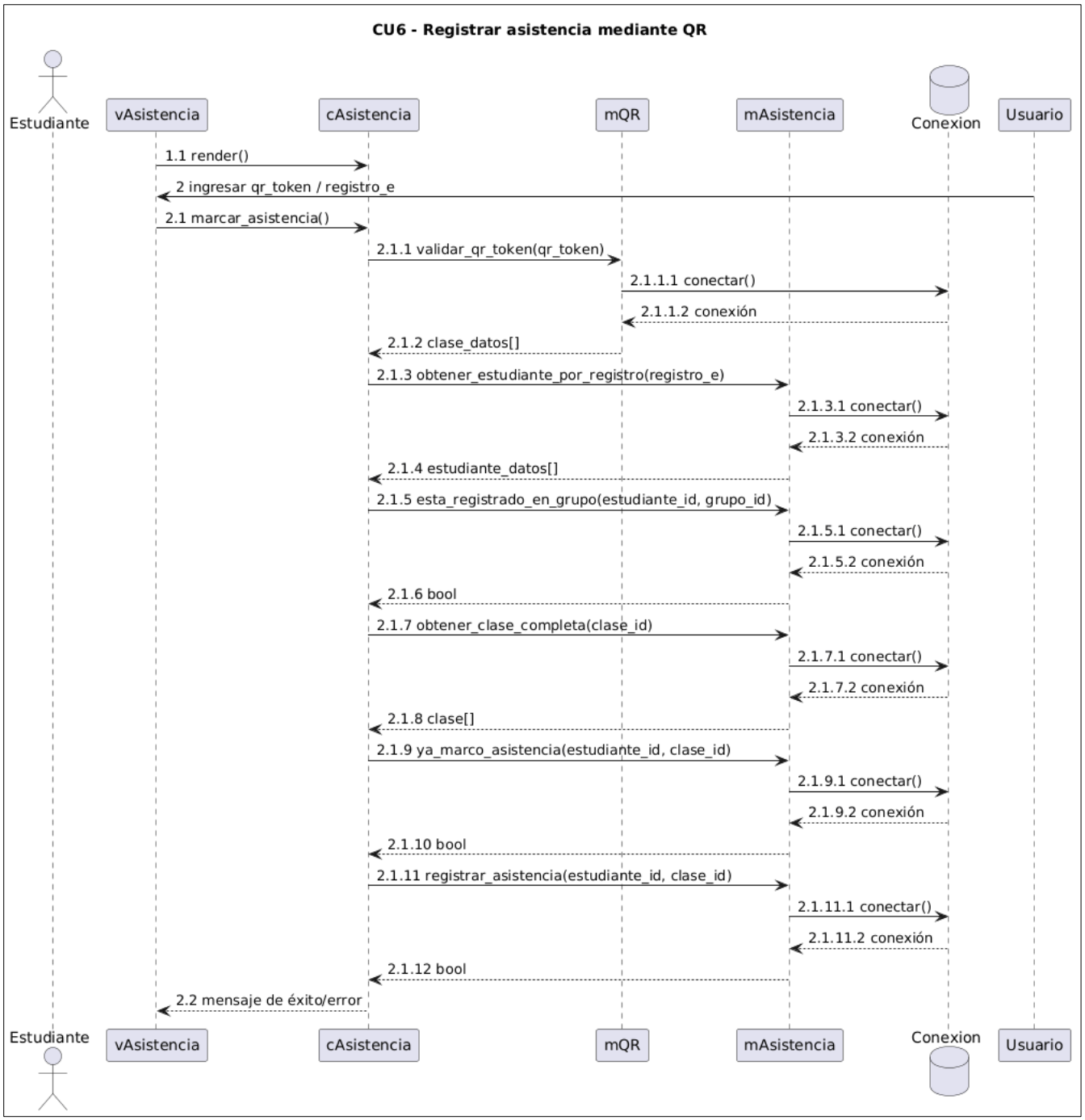
3.4.5. CU5 Generar código QR para la clase




CU5 - Generar código QR para la Clase
Docente
vQR
cQR
mQR
Conexion
Usuario
1 seleccionar gestión QR
1.1 render()
1.1.1 get_clases_por_docente(docente_id)
1.1.1.1 conectar()
1.1.1.2 conexión
1.1.2 clases[]
1.2 mostrar tabla
2 generar QR(clase_id)
2.1 generar_qr()
2.1.1 generar_qr_token(clase_id)
2.1.1.1 conectar()
2.1.1.2 conexión
2.1.2 qr_token
2.2 renderQR(token)
3 consultar QR de clase(clase_id)
3.1 obtener_qr_json()
3.1.1 get_qr_por_clase(clase_id)
3.1.1.1 conectar()
3.1.1.2 conexión
3.1.2 qr_data[]
3.2 mostrar QR
4 descargar QR
4.1 downloadQR()
Docente
vQR
cQR
mQR
Conexion
Usuario

3.4.6. CU6 Registrar asistencia mediante QR




CU6 - Registrar asistencia mediante QR
Estudiante
vAsistencia
cAsistencia
mQR
mAsistencia
Conexion
Usuario
1.1 render()
2 ingresar qr_token / registro_e
2.1 marcar_asistencia()
2.1.1 validar_qr_token(qr_token)
2.1.1.1 conectar()
2.1.1.2 conexión
2.1.2 clase_datos[]
2.1.3 obtener_estudiante_por_registro(registro_e)
2.1.3.1 conectar()
2.1.3.2 conexión
2.1.4 estudiante_datos[]
2.1.5 esta_registrado_en_grupo(estudiante_id, grupo_id)
2.1.5.1 conectar()
2.1.5.2 conexión
2.1.6 bool
2.1.7 obtener_clase_completa(clase_id)
2.1.7.1 conectar()
2.1.7.2 conexión
2.1.8 clase[]
2.1.9 ya_marco_asistencia(estudiante_id, clase_id)
2.1.9.1 conectar()
2.1.9.2 conexión
2.1.10 bool
2.1.11 registrar_asistencia(estudiante_id, clase_id)
2.1.11.1 conectar()
2.1.11.2 conexión
2.1.12 bool
2.2 mensaje de éxito/error